Refmon (Provenance Gate v2): A Privacy-Preserving Reference Monitor Combining Provenance Tagging, Role Contracts, and Cost-Sensitive Bayesian Log-Odds Calculus for Endpoint, Server, and Cluster Defense

Security Systems & High-Performance Computing Research Group
Department of Computer Science
refmon-research@example.edu

October 2026 (Specification v2.0; signal weights frozen since v1.0)

Abstract

Signature scanners and opaque machine-learning classifiers struggle against emerging social-engineering lures (such as ClickFix clipboard-paste execution), zero-day server remote code execution (RCE), prompt-injected AI coding agents, and stealth userland/kernel rootkits—while frequently exfiltrating sensitive command lines, URLs, and credentials into centralized telemetry lakes. We present **Refmon** (*Provenance Gate v2*, specification v2.0), a deterministic, platform-agnostic reference monitor that detects, contains, and explains attacks by judging *post-exploitation behavioral consequences* rather than exploit payloads. Refmon evaluates every causal "story" of system events along three orthogonal axes—**Origin** (untrusted external provenance), **Contract** (deviations from cryptographically signed per-role behavioral allowances), and **Effect** (high-risk capabilities)—combined via a monotone question-grouped log-likelihood ratio (LLR) calculus in natural log-odds units (*nats*) against Bayes-optimal cost thresholds ($\tau_{\text{hold}}, \tau_{\text{block}}$) and a mandatory *Contract Gate*. Raw payloads never leave an in-memory Observe boundary, where content is irreversibly converted into per-device HMAC-SHA256 digests after extracting scalar obfuscation features and confirming Bloom-filtered threat indicators. We further integrate a six-engine read-only cross-view rootkit sensor using multi-pass discrepancy intersection and an authenticated, unprivileged user-space command barrier that predicts a command's behaviour from its de-obfuscated text before it runs, together with a live audit dashboard suitable for shared university HPC clusters. Every structural change is evaluated under a pre-registration protocol: only on mutation variants generated from a random seed recorded *before* the change was written (nine seeds from v1.0 to v2.0). Across 46 golden scenarios spanning 17 attack families, 886 fresh mutation variants, a 16-scenario live attack/control suite, and over 1,860 automated unit and property tests, Refmon v2.0 achieves 100% golden accuracy, contains 98.4% of mutated attack variants (Wilson 95% CI $[97.1, 99.1]$) at a 0.0% neutral false-block rate ($[0.0, 2.4]$; 1.3% for v1.7 on the same variants), and passes adversarial privacy audits with zero raw content leaks and zero fingerprint reversals.

Keywords: Reference Monitor, Dynamic Provenance Tracking, Role Contracts, Bayesian Log-Likelihood Calculus, Differential Privacy, Cross-View Rootkit Detection, ClickFix Defense, HPC Cluster Security.

1 Introduction

Modern computing environments—ranging from user workstations and web application servers to shared multi-tenant university High-Performance Computing (HPC) clusters—face attack vectors that bypass traditional perimeter and signature defenses. Three trends exemplify this shift:

1. **Human-in-the-loop social engineering (ClickFix / TerminalFix / FileFix):** Rather than delivering a binary executable that anti-virus engines can quarantine, adversaries display fraudulent browser verification prompts or documentation snippets instructing the user to copy and paste a shell pipeline (e.g., `curl ... | sh` or base64-encoded PowerShell) directly into an interactive terminal [25]. Even when the user is deceived, a runtime barrier should intercept the pasted execution before harm occurs.
2. **Zero-day server RCE and AI agent hijacking:** Memory-corruption or deserialization flaws in web workers and Node.js runtimes (PHP2Shell, Node2Shell), as well as indirect prompt injections against autonomous AI coding agents [24], coerce trusted processes into spawning shells, dropping executable web shells, or reading application secrets.
3. **Stealth rootkits and defense tampering:** Userland preload rootkits (LD_PRELOAD, /etc/ld.so.preload), fileless memfd_create binaries, and loadable kernel module (LKM) rootkits hide processes, sockets, and modules from standard administrative utilities while attempting to blind monitoring daemons [21].

Existing Endpoint Detection and Response (EDR) tools suffer from two structural weaknesses when deployed

in academic or privacy-regulated environments. First, signature-driven rules fail on unseen zero-day exploits, while black-box neural anomaly detectors produce unexplainable alerts and remain vulnerable to baseline poisoning. Second, commercial EDR agents routinely record raw command-line arguments, HTTP bodies, file paths, and environment variables on disk or transmit them to cloud backends—violating European data-protection norms and exposing research secrets or credentials stored in shell histories.

Core Insight. Regardless of how an unknown vulnerability is exploited or how a user is socially engineered, the attacker must subsequently *act* to achieve objectives: spawn unexpected programs, write executable scripts into web roots, connect to unlisted external hosts, read credential stores, escalate privileges, encrypt files en masse, or tamper with security tooling. Those post-compromise actions violate the *role contract* of the acting process.

Contributions. This paper presents the design, mathematical foundations, implementation, and evaluation of **Refmon** (*Provenance Gate v2*, specification `v2.0`):

- **Privacy-First Six-Stage Reference Monitor (§3):** A platform-independent pipeline (`Observe` → `Tag` → `Link` → `Judge` → `Gate` → `Report`) enforcing an irreversible RAM-only privacy boundary via keyed HMAC-SHA256 digests, local Bloom-confirmed threat lists, ε-differential privacy, k-anonymity, and four-eyes pseudonym re-identification.
- **Monotone Bayesian Log-Odds Calculus & Contract Gate (§3.5–3.7):** A deterministic, training-free mathematical engine combining exponential provenance decay, Noisy-OR label fusion, Laplace-smoothed role surprisal, Good-Turing baseline maturity, dual-window Poisson/CUSUM rate tests, and Bayes minimal-risk thresholds ($\tau_{\text{hold}}, \tau_{\text{block}}$), including a proven monotonicity fix (`v1.3`) resolving a subtle jump discontinuity under fading labels, and trusted provenance modelled as lineage with a bounded lifetime (`v2.0`).
- **Six-Engine Cross-View Rootkit Sensor & Unprivileged Cluster Barrier (§3.8, §4.1):** Read-only user-space rootkit detection comparing high-level OS enumerations against low-level kernel/memory invariants across multiple passes, paired with an Ed25519-signed policy barrier, a token-authenticated local API, a de-obfuscating command-intent analyser, automatic file quarantine, and a clean light-theme telemetry interface operable without `root` privileges on shared Linux nodes.
- **Rigorous Reproducible Evaluation & Complete Proofs (§4, Appendices A–H):** Pre-registered variant-seed evaluation of every version from `v1.0` to `v2.0` across 46 golden scenarios, nine pre-registered variant seeds (886 mutated streams for `v2.0`), 16 live attack/benign scenarios, held-out command-evasion probes, and formal proofs of every mathematical property.

2 Background and Related Work

2.1 Reference Monitors and Information-Flow Provenance

Anderson's classical Reference Monitor concept [1] stipulates three requirements for an access mediation mechanism: it must be *tamperproof*, *always invoked*, and *small enough to be subject to complete analysis and tests*. Denning [2] formalized lattice-based information-flow control, demonstrating how security labels propagate from untrusted inputs to derived data and processes. Dynamic taint analysis systems such as TaintCheck [13] track byte-level taint at high instrumentation cost (5×–30× slowdown), whereas whole-system provenance monitors—including CamFlow [14], Sleuth [15], Holmes [16], and Unicorn [17]—track causal graphs at the OS process, file, and socket granularity.

Unlike graph-embedding approaches such as Unicorn [17] that hash multi-hop neighborhoods into high-dimensional sketches classified by unsupervised clustering (making individual false positives opaque to operators), Refmon aligns with Holmes [16] in correlating causal stories against high-level tactics, while introducing explicit *role contracts*, continuous half-life decay with persistent contract evidence, and exact cost-derived log-odds thresholds.

2.2 Mandatory Access Control and Process Contracts

Linux security frameworks such as SELinux [18], AppArmor [19], and Seccomp-BPF [20] enforce static per-binary mandatory access control (MAC) profiles. While effective for rigidly confined daemons, static MAC profiles cannot distinguish *provenance context*: an interactive shell (`bash`) must be allowed to execute `curl` and `sh` during normal developer workflows, yet the exact same shell executing command text pasted from an untrusted webpage (ClickFix) represents a severe breach. Refmon bridges MAC contracts and dynamic provenance via *provenance-conditioned contracts* (e.g., `external_content_exec: deny` and Design Decision D23, where role-permitted high-volume effects remain benign unless tainted by an external origin label).

2.3 Sequential Hypothesis Testing and Bayesian Fusion

Wald's Sequential Probability Ratio Test (SPRT) [3] and Page's Cumulative Sum (CUSUM) control chart [4],

proved minimax-optimal by Lorden [5], form the bedrock of statistical change-point detection in network intrusion detection [23]. Naive Bayes alert correlators sum log-likelihood ratios across all firing rules, which severely overcounts correlated indicators within the same behavioral dimension (e.g., three overlapping network-fetch heuristics firing on a single socket). Pearl's Noisy-OR model [7] handles independent causal pathways for label combination, while Refmon's *max-within-question, sum-across-questions* operator (§3.5) bounds within-axis correlation while preserving additive Bayesian evidence across orthogonal questions.

2.4 Cross-View Rootkit Detection

Host rootkit scanners such as `chkrootkit` and `rkhunter` rely heavily on static file-path signatures and known bad hashes, missing novel LKM hooks or memory-only implants. Cross-view diffing, pioneered by Strider GhostBuster [21] and formalized in virtual-machine introspection [22], compares a high-level API listing (susceptible to syscall or `readdir` hooking) against a lower-level structural view (e.g., kernel object trees in `/sys/module` or direct `kill(pid, 0)` / `/proc/[pid]/status` probes). Single-pass user-space scanners such as `unhide` suffer from transient race conditions when short-lived processes exit mid-scan; Refmon resolves this via m-pass discrepancy intersection integrated directly into its log-odds engine.

2.5 Privacy-Preserving Security Telemetry

Storing cleartext audit logs creates a high-value target for attackers and insiders. Cryptographic keyed hashing via HMAC [9] provides collision-resistant and preimage-resistant correlation tokens when keyed with a device-local secret, while Bloom filters [8] enable compact threat-indicator distribution. For cross-device fleet aggregation, ε-differential privacy [11] and k-anonymity [12] bound membership disclosure. Refmon combines all four mechanisms with cryptographic policy signing (Ed25519 [10]) and four-eyes dual-approver pseudonym escrow.

3 Methodology and Mathematical Formulation

3.1 System Architecture and Core Principles

Figures 1 and 2 depict the six-stage architecture of Refmon. Platform-specific sensors emit in-memory `RawEvent` tuples $(e_{\mathrm{id}}, t, d, \tau, a, o_{\mathrm{raw}}, \alpha_{\mathrm{raw}})$ across 13 canonical event types:

$$\mathcal{T} = \{\texttt{content_in}, \texttt{request_in}, \texttt{request_end}, \texttt{exec}, \texttt{net_connect}, \texttt{file_write}, \texttt{persist}, \texttt{asset_access}, \texttt{priv_change}, \texttt{security_tamper}, \texttt{user_input}, \texttt{admin_input}, \texttt{response_out}\}.$$

Each actor $a = (\mathrm{id}, \mathrm{parent}, r, \mathrm{interactive}, u)$ carries a workload role $r \in \mathcal{R}$ (16 built-in roles such as `interactive_shell`, `web_worker`, `node_app_server`, `package_installer`, `office`, `launcher`, `agent`, `browser`, `mail`, plus policy-extensible roles `backup_job`, `vuln_scanner`, `patch_tool`).

Two foundational invariants govern the entire pipeline:

Invariant 3.1 (Zero Raw Persistence)**.** No raw payload $c = o_{\mathrm{raw}}$.content or free-text attribute (`domain`, `host`, `ip`, `url_path`, `program`, `signer`) ever crosses the output boundary of `Observe`. Post-`Observe` `Event` records validate against a strict 21-key scalar/identifier allowlist $\mathcal{K}_{\mathrm{allow}}$.

Invariant 3.2 (Missing Information Neutrality)**.** Whenever a sensor lacks a capability (e.g., code-signature verification returns `signed = "unknown"` or a learned role baseline is not yet statistically mature), the dependent signal is omitted ($s_j = 0$) rather than treated as suspicious, and the capability gap is reported out-of-band via the `Heartbeat` monitor.

3.2 Stage 1 (`Observe`): Cryptographic Privacy Boundary and Obfuscation Features

Given raw content string $c \in \Sigma^*$ and a 256-bit device-local secret key K_{dev}, `Observe` first whitespace-normalizes $\tilde{c} = \mathrm{norm}(c)$ and computes its irreversible fingerprint:

$$f(c) = \texttt{"hmac:"} \,\|\, \mathrm{Hex}\Big(\mathrm{HMAC\text{-}SHA256}\left(K_{\mathrm{dev}}, \tilde{c}\right)\Big). \quad (1)$$

Before discarding c from RAM, if $\tau \in \{\texttt{exec}, \texttt{content_in}, \texttt{user_input}, \texttt{admin_input}\}$, `Observe` extracts three scalar obfuscation metrics over the character histogram $\{n_x\}_{x\in\Sigma}$ of c ($N_c = |c| > 0$, $p_x = n_x/N_c$):

$$H(c) = -\sum_{x\in\Sigma,\, p_x>0} p_x \log_2 p_x \quad (\text{bits/char}), \quad (2)$$

$$\rho_{\mathrm{enc}}(c) = \frac{1}{N_c}\sum_{i=1}^{N_c} \mathbf{1}\left[c_i \in \Sigma_{\mathrm{b64/hex}}\right], \quad (3)$$

$$r_{\mathrm{ws}}(c) = \max\left\{\ell \in \mathbb{N} : \exists\, i,\ c_{i..i+\ell-1} \in \{\text{' '}, \text{'\textbackslash t'}\}^{\ell}\right\}. \quad (4)$$

The `effect_hide` signal fires whenever $H(c) > H_{\max}$ (5.2 bits), $r_{\mathrm{ws}}(c) > r_{\max}$, or explicit `hidden_window` / `encoded_payload` flags hold, catching base64-encoded lures and ClickFix whitespace-padding attacks (where malicious commands are pushed off-screen via hundreds of spaces) without retaining a single character of c.

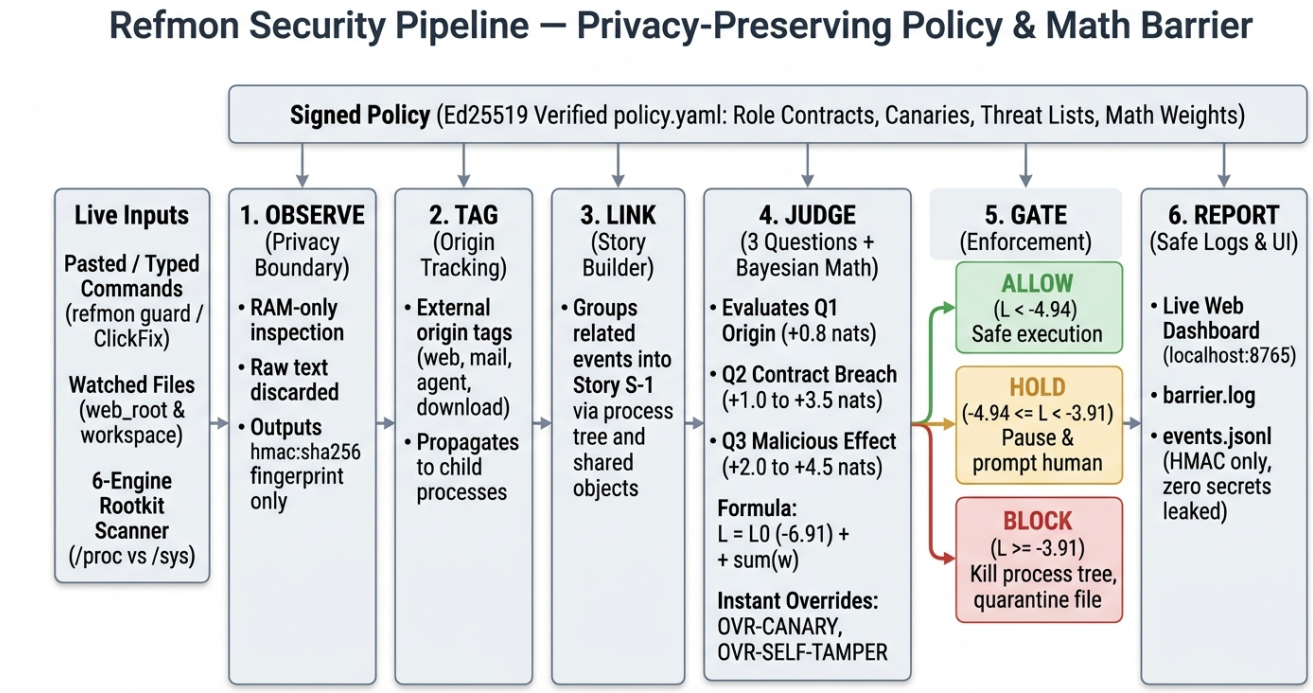


Figure 1: High-level architectural overview of the six-stage Refmon Security Pipeline governed by an Ed25519-signed policy (`policy.yaml`). Raw inputs are inspected exclusively in volatile memory inside Stage 1 (`Observe`) and immediately reduced to keyed `hmac:sha256` digests before entering provenance tagging, story linking, three-question Bayesian judging, containment gating, and privacy-safe reporting.

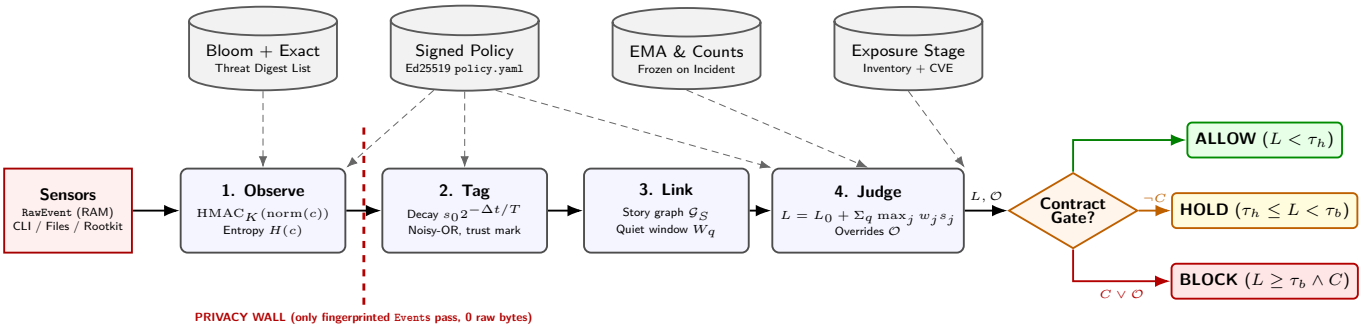


Figure 2: Formal TikZ vector schematic of Refmon's mathematical dataflow and shapes: rectangles denote deterministic functional stages, cylinders denote signed/stateful stores, the red dashed boundary marks the irreversible `Observe` cryptographic wall, and the diamond gate enforces the Contract Gate invariant ($L \geq \tau_{\text{block}} \wedge \neg C \wedge \mathcal{O} = \varnothing \implies$ HOLD).

Two-Stage Bloom Threat Confirmation. Threat indicators $\mathcal{D}_{\text{bad}}$ are stored as list-keyed HMAC digests $d_i = \text{HMAC-SHA256}(K_{\text{list}}, \tilde{v}_i)$ inside a bit-array Bloom filter $\mathcal{B}$ of size m bits and k_h double-hashed indices:

$$m = \left\lceil -\frac{n \ln p_{\text{fp}}}{(\ln 2)^2} \right\rceil, \qquad k_h = \max\left(1, \text{round}\left(\frac{m}{n} \ln 2\right)\right), \tag{5}$$

sized for $p_{\text{fp}} \leq 10^{-3}$. Any Bloom hit $\mathcal{B}.\text{might_contain}(d) = 1$ is treated strictly as a fast filter hint and verified against the exact digest set $\mathcal{D}_{\text{exact}}$ inside `Observe`, guaranteeing an operational threat-list false-positive rate of *strictly zero* (proved in Appendix F).

3.3 Stage 2 (`Tag`): Provenance DAG, Decay, and Noisy-OR Fusion

Untrusted external ingress ($\tau \in$ {`content_in`, `request_in`} with `source_class` in {`web`, `mail`, `removable`, `download`, `inbound_request`, `outbound_response`, `package`, `agent_input`} and not matching `trusted_sources`) mints a provenance label $\ell = (\text{id}, t_0, s_0 = 1, \text{persistent}, \mathcal{E}_{\text{contract}})$.

Exponential Half-Life Decay. At event timestamp $t \geq t_0$ ($\Delta t = t - t_0$), a non-persistent label with no recorded contract violation decays with policy half-life

$T_{1/2}$ (30 min):

$$s_\ell(t) = \begin{cases} s_0, & \text{if } \ell.\text{pers.} \vee \mathcal{E}_{\text{contract}}(\ell) \neq \varnothing, \\ s_0\, 2^{-\Delta t / T_{1/2}}, & \text{otherwise.} \end{cases} \tag{6}$$

Freezing decay once $\mathcal{E}_{\text{contract}}(\ell) \neq \varnothing$ (`v1.1`, decision A49) prevents an adversary from executing an initial contract breach, sleeping past $T_{1/2}$, and resuming post-exploitation unlabeled.

Edge Attenuation and Noisy-OR Combination. Across a causal edge e with attenuation $a_e \in (0, 1]$, child strength is $s_{\text{child}} = a_e \cdot s_{\text{parent}}$. When an actor or object carries multiple active labels $\{\ell_1, \ldots, \ell_m\}$ with strengths $s_1, \ldots, s_m \in [0, 1]$, their combined external-origin strength is fused via Pearl's Noisy-OR operator:

$$s_{\text{comb}}(t) = 1 - \prod_{i=1}^{m} \big(1 - s_{\ell_i}(t)\big). \tag{7}$$

When a labeled process writes an executable to a configured `web_root` or creates a startup entry (`persist`), the target object receives persistent = `true` along with the originating story's contract violations $\mathcal{E}_{\text{contract}}$, ensuring delayed execution days later (Scenario `S06`) inherits full contract evidence.

Order-Independent Label Hand-Over (`v1.7`, A64). External content whose fingerprint already carries labels hands *every* live label to the ingesting process (as an `exec` of labeled command text always did), not only the strongest one. Which label is strongest depends on the decay speed, so the earlier rule let the half-life change which stories events linked into; a randomised property test found a counterexample in which a faster-fading label produced a stronger story. Several labels combine by Eq. (7), so nothing is counted twice.

Trusted Provenance as Lineage (`v2.0`, A65). Trusted root sources (e.g. a university software portal) create no label (rule 6). Until `v1.7`, trust was therefore a property of a single story, whereas external provenance was a property of the process lineage: a later step of a signed installation further away than the quiet window W_q opened a new story without trust and could be blocked, although an external label in the same position would still have counted (finding F10). In `v2.0`, a non-channel process p that ingests trusted content at time t_p carries a *trust mark*, inherited by its children, which is live at time t iff

$$\begin{aligned} \text{trust}_p(t) = \mathbf{1}\Big[\, 2^{-(t-t_p)/T_{1/2}} \geq s_{\text{floor}} \Big] \\ \cdot\, \mathbf{1}\Big[\nexists\, \ell \in \text{ever}(p) : s_\ell(t) \geq s_{\text{floor}} \Big], \end{aligned} \tag{8}$$

where $s_{\text{floor}} = 0.02$ is the label alive floor and $\text{ever}(p)$ is every label the lineage of p has carried. The mark thus lives exactly as long as a non-persistent label created at t_p would ($T_{1/2} \log_2(1/s_{\text{floor}}) \approx 2.8$ h), is cancelled by any live external label in the lineage (even one faded below θ_{ori}), is not acquired by a channel role that ingests trusted content, and never passes to files or persistence items (Proposition G.1, Appendix G).

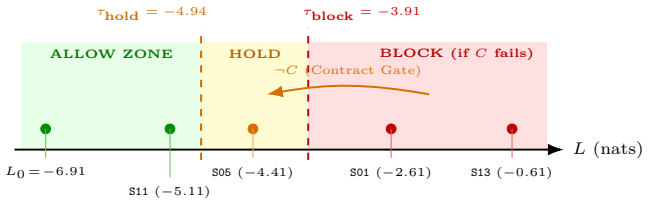


Figure 3: Bayesian log-odds decision geometry showing prior $L_0 = -6.91$ nats, cost thresholds $\tau_{\text{hold}} = -4.94$ and $\tau_{\text{block}} = -3.91$, benchmark trajectories (`S11` pasted benign echo, `S05` hand-typed fetch, `S01` ClickFix, `S13` web RCE), and the Contract Gate projection.

3.4 Stage 3 (`Link`): Causal Story Equivalence

`Link` maintains a dynamic partition of observed events into causal stories $\{S_1, S_2, \ldots\}$. Process-keyed actors merge along parent-child spawn trees (actor.parent) and shared labeled object fingerprints within the policy quiet window W_q (10 min), whereas channel roles (`browser`, `mail`) are excluded as merge keys (`non_process_keyed_roles`) so two unrelated downloads opened in the same browser never contaminate each other's stories.

3.5 Stage 4 (`Judge`): Three-Question Max-Sum LLR Calculus and Thresholds

For each story S, let $s_{\text{ori}}(S) = \max_{\ell \in \mathcal{L}(S)} s_\ell(t_{\text{last}})$ denote the strongest active provenance label. `Judge` evaluates three questions $\mathcal{Q} = \{\text{origin}, \text{contract}, \text{effect}\}$:

- Q_{origin} fails iff $s_{\text{ori}}(S) \geq \theta_{\text{ori}}$ (0.5), yielding signal `origin_external` with strength $s_{\text{ori}}(S) \in [\theta_{\text{ori}}, 1]$.
- Q_{contract} fails iff $\mathcal{S}_{\text{contract}}(S) \neq \varnothing$ (role violation, universal violation, unsigned external launch, or mature role surprisal), each with unit strength $s_j = 1$.
- Q_{effect} fails iff $\mathcal{S}_{\text{effect}}(S) \neq \varnothing$ after filtering role-allowed effects when $\neg Q_{\text{origin}}$ (Design Decision D23), each with unit strength $s_j = 1$.

A connection to a destination class marked `trusted` raises no `effect_fetch` for an unlabeled process, but since `v1.5` (A60) only when no live external label exists anywhere in the process's lineage: a downloaded script whose label has faded below θ_{ori} is still not excused by a popular trusted host.

Table 1: Frozen `v1.3` Policy Signal Weights (w_j in nats) and Cost Parameters ($\pi = 0.001$, $C_{\text{miss}} = 1000$, $C_{\text{fb}} = 20$, $C_{\text{hold}} = 5$, $h = 0.7$).

Axis	Signal Name	w_j	Signal Name	w_j
Origin	`origin_external`	0.8	—	—
Contract	`external_exec`	1.0	`write_other`	1.0
	`unsigned_launch`	1.5	`surprisal`	2.0
	`universal_violation`	2.0	`exec_violation`	2.5
	`connect_violation`	2.5	`asset_violation`	2.5
	`web_worker_exec`	3.0	`write_executable`	3.5
	`persist_violation`	3.5	`privilege_violation`	3.5
Effect	`asset_access`	1.5	`response_anomaly`	1.5
	`hide`	2.0	`fetch`	2.5
	`persist`	2.5	`exfil`	2.5
	`escalate`	2.5	`mass_change`	2.5
	`tamper`	3.0	`lateral`	3.5
	`credential`	3.5	`resp_anom_repeated`	3.5
	`threat_list_hit`	4.5	—	—

Max-Sum Log-Likelihood Ratio Calculus. Each signal j carries weight

$$w_j = \ln \frac{\mathbb{P}(j \mid \text{attack})}{\mathbb{P}(j \mid \text{benign})} \quad \text{(in nats)}, \tag{9}$$

either fixed from expert prior rates in the signed policy (Table 1) or re-estimated from labeled replay corpora with Laplace smoothing:

$$\hat{w}_j = \ln \frac{(a_j + 1)/(|\mathcal{A}| + 2)}{(b_j + 1)/(|\mathcal{B}| + 2)}. \tag{10}$$

Given prior attack probability $\pi \in (0, 1)$ ($\pi = 10^{-3} \implies L_0 = \ln \frac{\pi}{1-\pi} \approx -6.9068$ nats), the posterior log-odds of story S is:

$$L(S) = \ln \frac{\pi}{1-\pi} + \sum_{q \in \{\text{origin, contract, effect}\}} \max_{j \in \mathcal{S}_q(S)} \left(w_j \cdot s_j\right), \tag{11}$$

with empty maxima defined as 0, and posterior probability $\mathbb{P}(\text{attack} \mid S) = \sigma(L(S)) = (1 + e^{-L(S)})^{-1}$. Crucially, in `v1.3` (Decision A55), only Q_{origin} scales by $s_{\text{ori}}(S)$ while Contract and Effect signals take $s_j = 1$, guaranteeing that $L(S)$ is monotonically non-decreasing in label strength (Theorem C.1, Appendix C).

Cost-Based Decision Rule, Overrides, and Contract Gate. Let C_{miss} (1000), C_{fb} (20), and C_{hold} (5) denote the operational costs of a missed attack, a false block, and a human hold, and let $h \in (0, 1]$ (0.7) denote the probability that a hold stops a real attack. Bayes minimal-risk decision theory (proved in Appendix B) yields exact log-odds thresholds:

$$\tau_{\text{block}} = \ln \frac{C_{\text{fb}}}{C_{\text{miss}}} \approx -3.91, \quad \tau_{\text{hold}} = \ln \frac{C_{\text{hold}}}{hC_{\text{miss}}} \approx -4.94. \tag{12}$$

Preceding the calculus, four deterministic overrides $\mathcal{O}$ apply in strict priority order: (1) `OVR-SELF-TAMPER` $\mapsto$ `BLOCK`, (2) `OVR-CANARY` $\mapsto$ `BLOCK`, (3) `OVR-PROTECTED-ASSET` (labeled actor reading protected asset) $\mapsto$ `BLOCK`, and (4) `OVR-TRUSTED-ROOT` (an unlabeled story with an event that ingests trusted-root content or is performed by a process whose trust mark is live, Eq. (8)) $\mapsto$ `ALLOW`. When $\mathcal{O} = \varnothing$, the final decision $D(S)$ enforces the **Contract Gate** (Figure 3):

$$D(S) = \begin{cases} \texttt{BLOCK}, & \text{if } L(S) \geq \tau_{\text{block}} \ \wedge \ Q_{\text{contract}}(S), \\ \texttt{HOLD}, & \text{if } \big(L(S) \geq \tau_{\text{block}} \ \wedge \ \neg Q_{\text{contract}}(S)\big) \\ & \quad \vee \ \big(\tau_{\text{hold}} \leq L(S) < \tau_{\text{block}}\big), \\ \texttt{ALLOW}, & \text{if } L(S) < \tau_{\text{hold}}. \end{cases} \tag{13}$$

3.6 Contract Surprisal and Good-Turing Maturity

During contract learning or baseline counting for role r, let $n_{r,e}$ denote the observed count of event behavior kind e, $N_r = \sum_{e=1}^{K_r} n_{r,e}$ the total observations over K_r distinct seen kinds, and $n_{r,1} = |\{e : n_{r,e} = 1\}|$ the number of singleton kinds. With Laplace smoothing ($\alpha = 1$) reserving one pseudocount bin for all unseen behaviors ($K_r + 1$ bins total), the smoothed probability and Shannon surprisal are:

$$\hat{p}(e \mid r) = \frac{n_{r,e} + \alpha}{N_r + \alpha(K_r + 1)}, \quad I(e \mid r) = -\log_2 \hat{p}(e \mid r). \tag{14}$$

To uphold Invariant 3.2, neither learned contracts nor `contract_surprisal` ($I(e \mid r) \geq I_{\max} = 12$ bits) activate until the Good-Turing unseen probability mass falls below $\tau_{\text{mat}} = 0.01$:

$$\hat{p}_{\text{unseen}}(r) = \frac{n_{r,1}}{N_r} < \tau_{\text{mat}}. \tag{15}$$

3.7 Dual-Window Statistical Rate and Drift Detectors

Refmon monitors burst rates over both a short window (60 s) and a long window (1 h) so attackers cannot evade burst thresholds simply by spacing operations minutes apart:

1. **Poisson Upper-Tail Burst Test (`mass_change`, `lateral`, repeated `response_anomaly`):** Given EMA baseline rate $\hat{\lambda}$ and k distinct items in the active window, an alert fires iff

$$\mathbb{P}(X \geq k \mid \text{Pois}(\hat{\lambda})) = \sum_{i=k}^{\infty} \frac{e^{-\hat{\lambda}} \hat{\lambda}^i}{i!} < 10^{-6}. \tag{16}$$

2. **Standardized One-Sided CUSUM (`exfil` volume):** For role outbound byte volume x_t with role mean μ_r, standard deviation σ_r, standardized score

$z_t = (x_t - \mu_r)/\sigma_r$, slack $k_c = 0.5$, and threshold $h_c = 5.0$:

$$S_t = \max\big(0,\, S_{t-1} + z_t - k_c\big), \qquad \text{alert iff } S_t > h_c. \tag{17}$$

All EMA baselines $\hat{\lambda}_t = (1-\beta)\hat{\lambda}_{t-1} + \beta y_t$ freeze (frozen = `true`) whenever an incident on that process tree is open, preventing adversarial baseline poisoning. Since `v1.5` (A61) an incident ends once its process tree has been silent for the long window (1 h); before, an incident closed only when its tree later produced an allowed story, which a contained tree never does, so a single block froze every baseline indefinitely. A long-running daemon additionally forgets stories, idle processes, dead labels and rate state older than a retention horizon $r \geq \max(W_q, 1\,\text{h})$ (A62), never a story with a pending hold and never a process whose label or trust mark is still live, so its memory stays bounded without changing any decision on a retained story.

3.8 Six-Engine Cross-View Rootkit Calculus

To detect userland and kernel stealth implants without requiring root privileges, Refmon executes $m \geq 2$ passes separated by $\Delta\tau$ across six read-only engines: (E1) dynamic linker hijacking (`/etc/ld.so.preload`, `LD_PRELOAD`, `LD_AUDIT`, ephemeral `LD_LIBRARY_PATH`, deleted or ephemeral shared libraries), (E2) fileless execution and memory injection (`memfd:`, deleted `/proc/[pid]/exe` links, execution from `/dev/shm` or `/tmp`, non-zero `TracerPid`, anonymous `rwxp` segments), (E3) LKM cross-view (`/sys/module` \ `/proc/modules`) with kernel taint decoding (`/proc/sys/kernel/taint`: forced load or unload is scored, while the proprietary, out-of-tree and unsigned-module bits of GPU/DKMS drivers are recorded as context only), (E4) PID cross-view (`/proc` enumeration versus thread groups and own-UID ancestry, without sending signals), (E5) socket cross-view and sniffer check (`TCP: inuse` of `/proc/net/sockstat` versus the lines of `/proc/net/tcp`, and `IFF_PROMISC` interfaces), and (E6) an optional bridge to installed `unhide`, `chkrootkit` or `rkhunter`, where a missing tool adds no evidence.

Persistent cross-view discrepancies are isolated by multi-pass set intersection:

$$\Delta_{\text{pers}} = \bigcap_{r=1}^{m} \Big(\mathcal{V}_{\text{low}}^{(r)} \setminus \mathcal{V}_{\text{high}}^{(r)} \Big), \tag{18}$$

suppressing transient process/module churn exponentially in m (Appendix H). Confirmed discrepancies map to bounded log-odds evidence:

$$w_{\text{rk}}(d,m) = \min\Big(w_{\max},\; \ln \frac{p_{\text{tp}}}{p_{\text{race}}} + (m-1) \ln \frac{1}{q_{\text{persist}}} \Big), \tag{19}$$

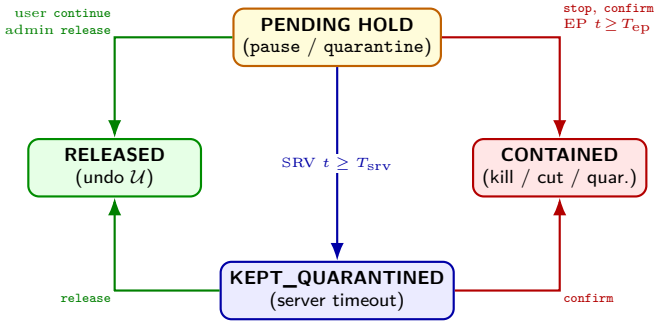


Figure 4: Dual endpoint vs. server hold state machine in `Gate`. Endpoint timeouts fail closed into containment, whereas unattended server holds preserve service stability in `KEPT_QUARANTINED` until an administrator decides.

The implementation treats passes as independent ($q_{\text{persist}} = p_{\text{race}} = 10^{-3}$, $p_{\text{tp}} = 0.99$, $w_{\max} = 8$ nats), so two passes already reach the cap; a cross-view finding counts with the larger of this weight and its base weight, and single-pass findings use fixed base weights (e.g. 2.2 nats for anonymous `rwxp` memory, 5.2 for `LD_PRELOAD`). Findings are emitted as privacy-redacted `SECURITY_TAMPER` or `EXEC` events directly into `Pipeline`.

3.9 Stages 5 & 6 (`Gate`, `Contain`, `Exposure`, `Report`)

Figure 4 illustrates `Gate`'s mode-aware enforcement (`observe` logs only; `warn` notifies without blocking; `block` enforces). Containment actions (`kill_process_tree`, `cut_network`, `quarantine_file` moving files with mode `0400` without deletion, `revert_persistence`, `restore_file`) target exclusively labeled entities and carry reversible undo tokens $\mathcal{U}$. When no process of a story is labeled, a hold or block falls back to the processes that produced the story's signals (`v1.2`, A51–A52; for blocks in stories with an external origin label since `v1.6`, A63, sparing the user's interactive shell), never to a channel role (`browser`, `mail`, `launcher`). Since `v2.0` (A66), a channel role that produced a signal by *starting* a program hands the fallback to the programs it started inside the story: the relay is never touched, but the program that acts is, so a hold whose only producer was a run dialog no longer pauses nothing. Concurrently, the `Exposure` stage matches local software inventories against CVE advisories, automatically escalating exposed or end-of-life roles to `block` mode with strict outbound network confinement (*virtual patching*) until patched.

For fleet-wide reporting, per-device aggregate counts add calibrated Laplace noise $\tilde{c} = c + \text{Lap}(1/\varepsilon)$, dashboard cohorts with fewer than $k = 5$ devices are suppressed, and pseudonym re-identification requires cryptographic four-eyes approval from two distinct human operators recorded on a hash-chained tamper-evident audit log.

Algorithm 1: End-to-End Refmon Event Processing (`Pipeline.process`)

Input: Sensor event $e_{\text{raw}} \in$ `RawEvent` $\cup$ `Event`, verified policy Π, device HMAC key K_{dev}.
Output: Emitted verdicts and containment actions $\mathcal{V} \subseteq$ `Verdict` $\cup$ `Action`.
$e \leftarrow \textsc{Observe}(e_{\text{raw}}, \Pi, K_{\text{dev}})$ ▷ Compute $\text{HMAC}_{K_{\text{dev}}}$, $H(c)$, Bloom+Exact check; drop raw content
$p_{\text{known}} \leftarrow (e.\text{actor.parent} \in \text{Tagger.procs})$
$\text{tg} \leftarrow \textsc{Tag}(e, \Pi)$ ▷ Apply propagation rules 1–7, decay $2^{-\Delta t / T_{1/2}}$, Noisy-OR
$(\mathcal{S}, \mathcal{F}) \leftarrow \textsc{ExtractSignals}(e, \text{tg}, \Pi, \text{Rates}, \text{Baseline})$
$\text{rec} \leftarrow \textsc{Link}(\text{tg}, p_{\text{known}}, \mathcal{S}, \mathcal{F}, \Pi)$ ▷ Assign/merge story within quiet window W_q
$J \leftarrow \textsc{Judge}(\text{rec}, \text{Tagger}, \Pi)$ ▷ Algorithm 2
Rates.mark_incident(root(e.actor), J.decision $\neq$ `ALLOW`, e.ts) ▷ Freeze baselines; end incidents silent for 1 h
if $\{s \in J.\text{signals} : s.\text{question} = \text{contract} \wedge \neg s.\text{carried}\} \neq \varnothing$ **then**
    Freeze decay and attach contract evidence to active labels $\ell \in$ rec.label_ids
$\mathcal{V} \leftarrow \textsc{EmitVerdictIfChanged}(\text{rec}, J)$
if Gate $\neq$ `None` **then**
$\mathcal{V} \leftarrow$ Gate.tick(e.ts) $\|\, \mathcal{V} \,\|$ Gate.handle(rec, J, e.ts)
return $\mathcal{V}$

Algorithm 2: Monotone Question-Grouped Bayesian Judge (`v2.0`)

Input: Story record rec, provenance tagger $\mathcal{T}$, signed policy Π.
Output: Story judgement $J = (\text{decision}, L, \text{failed}, \text{override}, \text{gated})$.
$s_{\text{ori}} \leftarrow \max_{\ell \in \text{rec.label_ids}} \mathcal{T}.\text{strength}(\ell, \text{rec.last_ts})$; $\text{labeled} \leftarrow (s_{\text{ori}} \geq \Pi.\theta_{\text{ori}}) \wedge \Pi.\text{enabled}(\texttt{R-ORI})$
Filter $\mathcal{S} \leftarrow \{s \in \text{rec.signals} : (\neg s.\text{allowed} \vee \text{labeled}) \wedge \Pi.\text{rule_enabled}(s) \wedge s \notin \Pi.\text{exceptions}\}$
Append persistent carried contract signals from $\{\ell \in \text{rec.label_ids} : s_\ell \geq \Pi.\theta_{\text{ori}}\}$
Check priority overrides $\mathcal{O}$: `self_tamper` $\succ$ `canary` $\succ$ `protected_asset` $\mapsto$ `BLOCK`; unlabeled `trusted` (source event or live trust mark) $\mapsto$ `ALLOW`
if $\mathcal{O} \neq$ `None` **then return** forced override decision with audit trace
$\mathcal{E} \leftarrow \{(s.\text{question}, s.\text{name}, 1.0) : s \in \mathcal{S}\}$ ▷ `v1.3`: unit strength for Contract & Effect
if labeled **then** $\mathcal{E} \leftarrow \mathcal{E} \cup \{(\text{origin}, \texttt{origin_external}, s_{\text{ori}})\}$
$L \leftarrow \ln \frac{\Pi.\pi}{1-\Pi.\pi} + \sum_{q \in \{\text{ori,con,eff}\}} \max_{(q,j,s_j) \in \mathcal{E}} (\Pi.w_j \cdot s_j)$
$\tau_b \leftarrow \ln(\Pi.C_{\text{fb}} / \Pi.C_{\text{miss}})$; $\tau_h \leftarrow \ln(\Pi.C_{\text{hold}} / (\Pi.h \cdot \Pi.C_{\text{miss}}))$; $C_{\text{fail}} \leftarrow \exists s \in \mathcal{S} : s.\text{question} = \text{contract}$
if $L \geq \tau_b \wedge C_{\text{fail}}$ **then return** (`BLOCK`, L, gated = `false`)
else if $L \geq \tau_b \wedge \neg C_{\text{fail}}$ **then return** (`HOLD`, L, gated = `true`) ▷ Contract Gate
else if $L \geq \tau_h$ **then return** (`HOLD`, L, gated = `false`)
else return (`ALLOW`, L, gated = `false`)

Algorithm 3: Multi-Pass 6-Engine Read-Only Rootkit Cross-View Sensor

Input: Pass count $m \geq 2$, interval $\Delta\tau$, own UID u, signed policy Π.
Output: Structured `RootkitScanReport` with posterior L_{rk} and decision D_{rk}.
$\mathcal{F}_{\text{single}} \leftarrow \textsc{ScanPreloadAndMemfdAndMaps}(u) \cup \textsc{ScanTaintAndPromisc}()$
for $r = 1 \ldots m$ **do**
    $\Delta^{(r)}_{\text{mod}} \leftarrow \textsc{SysfsModules}() \setminus \textsc{ProcModules}()$ ▷ Engine 4: Hidden LKM check
    $\Delta^{(r)}_{\text{pid}} \leftarrow \textsc{DirectOwnPidProbe}(u) \setminus \textsc{ProcReaddirPids}()$ ▷ Engine 6: Hidden PID check
    if $r < m$ **then** sleep($\Delta\tau$)
$\Delta^{*}_{\text{mod}} \leftarrow \bigcap_{r=1}^{m} \Delta^{(r)}_{\text{mod}}$; $\Delta^{*}_{\text{pid}} \leftarrow \bigcap_{r=1}^{m} \Delta^{(r)}_{\text{pid}}$ ▷ Suppress transient churn
Map $\mathcal{F} \leftarrow \mathcal{F}_{\text{single}} \cup \textsc{Findings}(\Delta^{*}_{\text{mod}}, \Delta^{*}_{\text{pid}})$ to log-odds $L_{\text{rk}} = L_0 + \sum_{f \in \mathcal{F}} w_{\text{rk}}(f, m)$
Evaluate $D_{\text{rk}} \leftarrow \textsc{Decide}(L_{\text{rk}}, \tau_b, \tau_h, \text{override} =$ `SELF_TAMPER if` $\exists f.\text{self_tamper})$
return `RootkitScanReport`$(\mathcal{F}, L_{\text{rk}}, D_{\text{rk}})$

4 Experiments, Algorithms, and Implementation Tests

4.1 Reference Implementation and Signed Policy Governance

Refmon is implemented in approximately 12,400 lines of strict, type-checked Python 3.12 (plus 10,200 lines of tests) (`pydantic` v2 frozen schemas, `cryptography` Ed25519 signatures, zero runtime machine-learning dependencies). Every threshold, weight, role contract, canary, and rate parameter resides in a canonical JSON-serialized, Ed25519-signed `policy.yaml`; startup and live reload reject any unsigned or tampered policy (`PolicyRejected`) and retain the last verified state. A periodic `SelfTest` scheduler clones the live pipeline via `pipeline.probe()` (preserving live disabled rules and weights while isolating outputs) to inject synthetic health stories and raise immediate alerts if any enabled protection fails to reach its expected decision.

4.2 Core Pipeline and Scanner Algorithms

Algorithms 1, 2, and 3 specify the exact deterministic execution of the Refmon event loop, the monotone Bayesian `Judge`, and the multi-pass cross-view rootkit sensor. Listing 1 shows the core unprivileged command-barrier enforcement loop from `src/refmon/live.py`.

Listing 1: Core unprivileged user-space command barrier loop (excerpted from `src/refmon/live.py`).

```
story_id, j = self._feed_raw_events(raw_events)  # Observe ->
     Tag -> Link -> Judge -> Gate
lo = float(j.log_odds if j.log_odds is not None else 0.0)
blocked_by_barrier = (
    self.policy.mode is Mode.BLOCK
    and j.decision in (Decision.BLOCK, Decision.HOLD)
)
if execute and not blocked_by_barrier and j.decision is
     Decision.ALLOW:
    proc = subprocess.run(cmd, shell=True,
     cwd=str(self.workspace_dir),
                          capture_output=True, text=True,
     timeout=5.0, check=False)
```

4.3 Golden Catalog and Seeded Mutation Robustness

Strict Pre-Registration Protocol. All weights (Table 1) and structural rules were fixed prior to catalog evaluation, and the weights have not changed since. Every later structural change needs a reason that names no scenario and is evaluated only on variants of a generator seed drawn with `secrets.randbelow` and recorded *before* the change was written; the old and new versions replay the identical variant file (stream hashes checked), and the seed is retired afterwards because its findings motivate the next change. Robustness across the first four specification tags (`v1.0` through `v1.3`) is measured on 879 event-stream variants mutated along six orthogonal dimensions (delivery channel, launcher entry, obfuscation, network destination, process lineage depth, and inter-event timing) generated from pre-registered seed `654514256` (SHA-256 stream digest `5c3fcbeb...`), guaranteeing identical input streams across versions.

Table 2 summarizes the progression from `v1.0` to `v1.3`. All 46 golden scenarios across all 17 attack families reach their exact expected decisions (100%). Across the 633 malicious variants, fixing hold containment on unlabeled signal-producing processes (`v1.1`–`v1.2`) eliminated all 69 "holds that paused nothing" and all 11 "blocks that acted on nothing," raising active containment from 86.3% to 98.7% ($p = 3.3 \times 10^{-24}$). Enforcing monotonicity in `v1.3` converted 19 faded-label holds into decisive blocks ($p = 3.8 \times 10^{-6}$ vs. `v1.2`) while keeping the false-block rate on neutral benign mutations unchanged at 2.0% (3/151).

Rounds `v1.4` to `v2.0`. Table 3 lists the four pre-registered rounds that followed. The `v1.7` round exposed the last benign false blocks: three timing variants of a signed installation from a trusted software portal whose steps lay further apart than W_q (finding F10). On the fresh seed `171139148` (886 variants), `v2.0` turns both remaining neutral false blocks of `v1.7` into allows and three false holds of a trusted deployment into allows, so the neutral benign outcome moves from 132/24/2 (allow/hold/block, 1.3% [0.3, 4.5] false blocks) to 137/21/0, every remaining hold being one the specification expects (`S49`, `S50`), and no benign variant ends worse than its unmodified file. No malicious variant changed: 625/635 are contained by both versions (98.4%), 314/324 (96.9% [94.4, 98.3]) on the subset the seed decides. The exact McNemar test on the changed benign pairs $(0, 5)$ gives $p = 0.0625$: consistent with the purpose of A65, but five variants are too few for significance on their own. A66 had nothing to act on in this seed (`v1.7` produced no hold that paused nothing); its evidence is its unit tests and a randomised containment property test, which rejected a first, too wide draft. Malicious containment stayed between 98.1% and 99.1% across the four seeds, with the neutral false-block rate falling from 2.0% (`v1.3`) to 0.0% (`v2.0`).

4.4 Live Cluster Barrier, 16-Scenario Suite, and Rootkit Tests

Figure 5 illustrates how Refmon operates on shared Linux HPC nodes where researchers lack `root` privileges. The background daemon (`refmon serve`) bootstraps an Ed25519 keypair, verifies `policy.yaml` in `block` mode, watches user-space staging directories every 400 ms, and exposes a minimal white-theme local telemetry dashboard. Because every user of a shared node can reach a localhost port, every API call must carry a random per-daemon token kept in the owner-only (`0700`) work directory, requests whose `Host` header is not local are refused (DNS rebinding), and Observe fingerprints are keyed with a random per-device key stored next to it. Table 5 reports the live 16-scenario verification suite (16/16 = 100% expected verdicts): ClickFix lures are halted prior to shell execution (`executed=False`), dropped PHP web shells are automatically quarantined within < 500 ms with `0400` permissions, 55-file ransomware bursts trigger `mass_change` blocks, canary/tamper tripwires trigger instant overrides, and benign local or PyPI operations execute cleanly with zero false alarms. In synthetic and live host tests (13 unit tests in `test_rootkit.py`), the six-engine rootkit scanner suppresses benign NVIDIA/out-of-tree GPU kernel taints ($L = -6.91 \mapsto$ `ALLOW`), holds lone JIT-like anonymous `rwxp` regions ($L = -4.71 \mapsto$ `HOLD`), and blocks `LD_PRELOAD`, `memfd`, hidden LKMs, and hidden PIDs ($L \geq -2.51 \mapsto$ `BLOCK`).

Predicting a Command's Behaviour Before It Runs. The barrier cannot observe a command that has not run, so it predicts the events the pipeline would observe from the command text and judges them with the same contracts and weights: outbound hosts (URLs, `/dev/tcp`, `socat`, `nc`, socket calls in inline interpreters, reverse and dynamic SSH tunnels), reads of key material (`.aws`, `.kube`, `.netrc`, private SSH keys, but not a key *used* via `ssh -i`), writes to startup files, schedulers and git hooks, tampering with Refmon, security services or shell history, privilege changes (including containers given the host), preload libraries from scratch locations, whole-tree deletion or encryption, and sending the home directory or the root to a host. Since `v2.0` (A67) the text is analysed twice, as written and after undoing the shell's own word-hiding (empty quotes or a backslash inside a word, `${IFS}`, `$'\x..'` escapes, quotes around one word, variables assigned earlier on the line); removing what the shell itself removes never adds a behaviour the shell would not see, and hiding a word that way is itself the `hide` effect. Table 4 reports the probe sets typed locally (no origin label, so a refusal needs at least a hold). The held-out set was written after the analyser and run once; its five misses and one false alarm were fixed afterwards, so all 69 probes now serve as regression coverage, not as a measurement.

Table 2: Robustness Evaluation Across Specification Versions `v1.0`–`v1.3` on 46 Golden Scenarios and 879 Seeded Variants (Seed `654514256`), with Wilson 95% Confidence Intervals [26] and Paired Exact McNemar Tests [27].

Evaluation Metric / Benchmark Partition	**`v1.0` (Frozen)**	**`v1.1` (A46–A50)**	**`v1.2` (A51–A54)**	**`v1.3` (Monotone)**
Golden Catalog Scenarios Matching Expected Decision ($N = 46$)	46/46 (100.0%)	46/46 (100.0%)	46/46 (100.0%)	46/46 (100.0%)
Malicious Variants ($N = 633$)				
Blocked outright	425 (67.1%)	467 (73.8%)	467 (73.8%)	486 (76.8%)
Effective Hold (process paused / file quarantined)	121 (19.1%)	157 (24.8%)	157 (24.8%)	139 (22.0%)
Contained = Block + Effective Hold [Wilson 95% CI]	86.3% [83.4, 88.7]	98.6% [97.3, 99.3]	98.6% [97.3, 99.3]	98.7% [97.5, 99.4]
Caught = Block + Any Hold [Wilson 95% CI]	97.2% [95.6, 98.2]	98.6% [97.3, 99.3]	98.6% [97.3, 99.3]	98.7% [97.5, 99.4]
Uncontained: Hold that paused nothing	69	0	0	0
Uncontained: Allowed malicious variants (Gaps G1–G3)	18	9	9	8
Blocks that acted on nothing (unlabeled fallback gap)	11	9	0	0
Independent Seed-Decided Malicious Subset ($N = 322$)				
Contained (Block + Effective Hold)	269/322 (83.5%)	313/322 (97.2%)	313/322 (97.2%)	314/322 (97.5%)
Neutral Benign Variants ($N = 151$, False-Alarm Test)				
Allowed / Held / False-Blocked	126 / 22 / 3	126 / 22 / 3	126 / 22 / 3	125 / 23 / 3
Neutral False-Block Rate [Wilson 95% CI]	2.0% [0.7, 5.7]	2.0% [0.7, 5.7]	2.0% [0.7, 5.7]	2.0% [0.7, 5.7]
Paired McNemar Exact Test vs. `v1.3` (Malicious, $N = 633$)				
Contained: (only baseline, only `v1.3`), exact p-value	$(0, 79)$, $p = 3.3\times10^{-24}$	$(0, 1)$, $p = 1.0$	$(0, 1)$, $p = 1.0$	—
Blocked: (only baseline, only `v1.3`), exact p-value	$(0, 61)$, $p = 8.7\times10^{-19}$	$(0, 19)$, $p = 3.8\times10^{-6}$	$(0, 19)$, $p = 3.8\times10^{-6}$	—

Table 3: Pre-Registered Rounds `v1.4`→`v2.0`. Each round replays one fresh seed's variants through the version before and after the change. "Contained" = block + hold that paused, quarantined or killed something; Wilson 95% intervals; neutral benign variants carry no mutation the specification forbids.

Round	**Seed**	**Structural change (design decision; motivating finding)**	**Contained, before → after**	**Neutral false blocks**	**Benign changed**
`v1.4`→`v1.5`	`833754500`	lineage-aware destination trust, incidents end, bounded state (A60–A62; F2)	630 → 631/637, 99.1% [98.0, 99.6]	1/158 (0.6%)	none
`v1.5`→`v1.6`	`545728926`	labeled block with no labeled process acts on its producers (A63; F11)	623 → 624/636, 98.1% [96.7, 98.9]	1/161 (0.6%)	none
`v1.6`→`v1.7`	`616687759`	every live label handed over (A64; property test)	623 → 623/634, 98.3% [96.9, 99.0]	3/154 (1.9%)	none
`v1.7`→`v2.0`	`171139148`	trust as lineage, channel roles relay (A65, A66; F10)	625 → 625/635, 98.4% [97.1, 99.1]	**0/158** (**0.0**% [0.0, 2.4])	5, all improved

Table 4: Command-Barrier Probes Typed Locally (`mode=block`). Bold: the held-out measurement.

Probe set	**`v1.7`**	**`v2.0` first run**	**`v2.0` now**
23 attacker behaviours (`test_intent.py`)	23/23	23/23	23/23
20 evasions, written before A67	11/20	20/20	20/20
20 attacks, held out	13/20	**15/20**	20/20
23 + 14 benign look-alikes (false alarms)	1	0	0
15 benign, held out (false alarms)	1	**1**	0

Table 5: Live Barrier 16-Scenario Verification Suite (`mode=block`).

Scenario Type	**Origin / Role**	**Q-Fails**	L **(nats)**	**Verdict**
ClickFix Paste (`curl|sh`)	`web` / `shell`	3/3	−0.61	`BLOCK`
Obfuscated Base64 Mail	`mail` / `shell`	3/3	−3.11	`BLOCK`
AI Prompt-Injection Abuse	`agent_in` / `agent`	3/3	−0.11	`BLOCK`
Protected Credential Theft	`mail` / `shell`	OVR	−1.61	`BLOCK`
Server Web Worker RCE	`req_in` / `web`	3/3	−0.61	`BLOCK`
Web Shell Drop (`.php`)	`req_in` / `web`	3/3	−0.11	`BLOCK+QUAR`
Ransomware 55-File Burst	local / `shell`	2/3	−2.41	`BLOCK`
Decoy Canary Credential	`web` / `shell`	OVR	−1.61	`BLOCK`
SUID / `sudo` Escalation	`dl` / `shell`	3/3	−2.61	`BLOCK`
Crontab / `.bashrc` Persist	`web` / `shell`	3/3	−2.61	`BLOCK`
Security Tamper (`pkill`)	local / `shell`	OVR	−1.91	`BLOCK`
Supply-Chain Post-Install	`pkg` / `installer`	3/3	+1.89	`BLOCK`
Unverified Mirror Fetch	local / `shell`	1/3	−4.41	`HOLD`
Benign Local `ls -la`	local / `shell`	0/3	−6.91	`ALLOW`
Benign Pasted `echo`	`web` / `shell`	2/3	−5.11	`ALLOW`
Trusted PyPI Download	`dl` / `installer`	OVR	−6.91	`ALLOW`

4.5 Adversarial Privacy Audit and Test Suite Summary

Running `refmon audit` executes an active red-team privacy audit across all generated verdicts, reports, stories, buffered events, learning proposals, and golden files:

- **Cleartext Leakage:** 264 output artefacts scanned → 0 **raw content findings**.
- **Dictionary HMAC Attack:** 349 fingerprints attacked across 2,119 common command/URL guesses without $K_{\text{dev}} \to 0$ **reversals** (whereas a positive control given K_{dev} reverses 6/6, confirming attack-harness validity).
- **Four-Eyes Escrow:** 10 single-approver re-identification attempts → 0 **permitted**; dual-approver

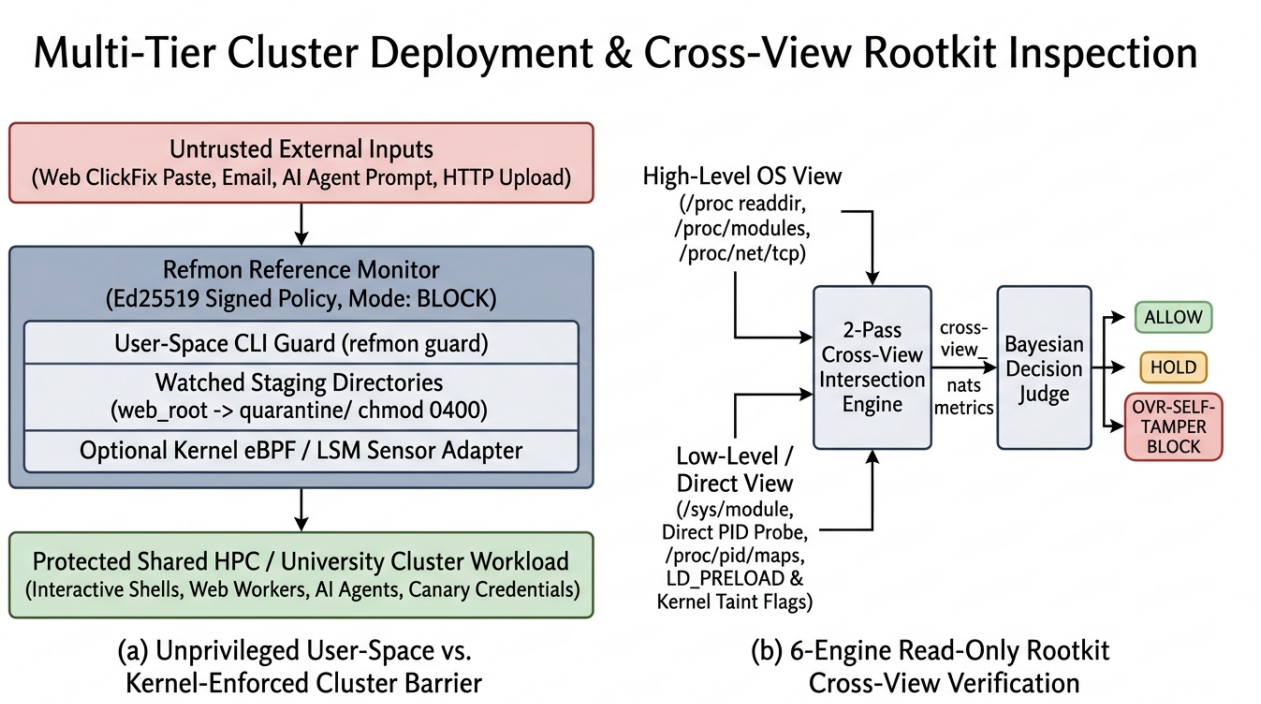


Figure 5: Multi-tier deployment architecture and host integrity verification: (a) unprivileged user-space command barrier (`refmon guard`) and watched staging directories (`web_root` → `quarantine/` mode `0400`) alongside optional kernel eBPF/LSM sensors; (b) six-engine two-pass cross-view rootkit sensor feeding bounded log-odds into the Bayesian decision judge.

requests succeed with hash-chain integrity verified (`audit chain intact = True`).

- **Automated Verification:** Across unit, Hypothesis property-based, golden replay, variant-matrix, rootkit, command-intent, CLI, and live HTTP tests, over 1,860 automated tests pass.

5 Discussion

5.1 Why Consequence-Oriented Contracts Generalize to Unseen Zero-Days

Exploit techniques evolve rapidly, yet post-compromise objectives remain tightly constrained by operating-system semantics. Whether a web worker is compromised through a classic PHP file inclusion, a novel HTTP/3 heap overflow, or an unsafe Node.js deserialization gadget (`S13`, `S25`, `S26b-c`), spawning an interactive shell or writing an executable script into a web root violates the invariant contract of that worker role (`may_exec: []`). Simultaneously, evaluating Origin and Effect alongside Contract prevents rigid MAC brittleness: when a developer copies a benign diagnostic command (`echo` or `git status`, Scenario `S11`) from documentation, Origin (+0.8 nats) and Contract (+1.0 nats) fire, reaching $L = -5.11$ nats—safely below $\tau_{\text{hold}} = -4.94$ nats and avoiding alert fatigue.

5.2 Discovery and Resolution of the Non-Monotonicity Flaw (Finding F8)

Pre-registered mutation testing revealed a subtle mathematical pathology in versions prior to `v1.3`. Earlier versions scaled Contract and Effect signals by s_{ori} whenever $s_{\text{ori}} \geq \theta_{\text{ori}} = 0.5$, while reverting to unit strength (1.0) when $s_{\text{ori}} < 0.5$ so unlabeled local breaches could still be caught. Consequently, a provenance label decaying from $s_{\text{ori}} = 0.70$ to 0.49 abruptly *increased* a Contract signal's contribution from $0.70w_j$ to $1.00w_j$—violating evidence monotonicity. Decoupling Contract and Effect strengths ($s_j \equiv 1$) from Origin's continuous decay in `v1.3` restored global monotonicity (proved in Appendix C) and upgraded 19 faded-label attack variants from `HOLD` to `BLOCK` ($p = 3.8 \times 10^{-6}$) without increasing neutral false blocks.

5.3 Trust as Lineage and the Last False Blocks (Finding F10)

The false blocks that remained through `v1.7` came from an asymmetry, not from a weight: external provenance lived in the process lineage and survived the story quiet window, whereas trusted provenance lived in a single story and did not. A signed installer that wrote its startup entry more than ten minutes after downloading its package was therefore judged as if nothing trusted had happened.

Carrying the trusted-root override across stories had been rejected earlier because it widens an *absolute* allow that a compromised signed updater could use. `v2.0` resolves the tension by giving trust exactly the lifetime of a label and strictly fewer paths (not acquired by channel roles, no files or persistence items, cancelled by any live external label, outranked by every blocking override), so the allow is widened in time only as far as an external label reaches, and in no new direction (Proposition G.1).

5.4 Why Refmon Does Not Learn Its Decisions

Reinforcement learning is a poor fit for the decision itself. Real attacks are rare, so rewards are sparse and delayed; an agent that explores by killing or quarantining processes on a shared node causes the harm it should prevent; feedback channels ("false alarm", "confirmed") can be poisoned by the attacker; and a learned policy cannot state the exact evidence behind a verdict, which every Refmon verdict does in nats. Learning has two safe places outside the decision path: re-estimating weights offline from labeled replay counts (Eq. (10)), applied only after human approval, and an adversarial search over the replay lab that looks for event sequences the deterministic monitor allows (Section 6).

5.5 Documented Architectural Boundaries and Residual Gaps

Refmon explicitly documents four fundamental limits rather than masking them behind overfitted weights:

1. **Unobserved Out-of-Band Channels (`S05`):** If a user manually types a malicious command viewed on a separate phone screen or video call, no digital `content_in` event occurs on the host. With Q_{origin} absent, `effect_fetch` (+2.5 nats) raises L to $-4.41 \geq \tau_{\text{hold}}$, pausing the process for human confirmation (`HOLD`) rather than silently allowing or blindly blocking.
2. **In-Contract Data Abuse (`S22`):** SQL injection that reads records strictly over the web application's already-permitted database socket violates no process or network contract; Refmon catches such attacks only when response sizes trigger `response_anomaly_repeated` (`S27`), outbound volume trips CUSUM (`exfil`), or decoy records trigger `OVR-CANARY` (`S08`).
3. **Residual Mutation Gaps (`G1`–`G5`):** Exactly 10/635 (1.6%) mutated malicious streams remain uncontained in `v2.0` on seed `171139148`: three actors with no external lineage at all connecting to a broadly trusted host class (`G1`; closing it means a trusted class excuses only roles whose contract lists it, a policy decision with a false-alarm cost), five ultra-slow ransomware mutations modifying 55 files minutes apart (`G2`–`G3`), and two slow memory-leak mutations whose oversized responses are minutes apart (`G4`–`G5`; one ends 0.02 nats below τ_{hold} because its request label has faded). None is closed by moving a weight or a threshold, which would fit the rule to the seed; the slow cases need a baseline measured in a pilot.
4. **User-Space vs. Kernel Deployment:** In unprivileged user space on shared HPC nodes, Refmon mediates commands routed through `refmon guard`, watched staging directories, and user-readable `/proc`/`/sys` views. Full non-bypassability against a local attacker who invokes raw syscalls directly requires administrative deployment of kernel-level sensors.

6 Future Work

Six directions extend Refmon toward production campus and HPC deployments:

1. **Kernel-Enforced Linux `eBPF` LSM and `fanotify` Adapters (Phase 8):** Implementing non-bypassable kernel sensors hooking `bprm_check_security`, `socket_connect`, and `file_permission` behind Refmon's frozen `Sensor.emit` and `Enforcer.act` interfaces, enabling synchronous kernel-level `HOLD`/`BLOCK` across all system processes without modifying a single line of core pipeline code.
2. **Distributed HPC Cluster & Batch-Scheduler Provenance:** Propagating HMAC provenance labels and role contracts across multi-node Slurm/PBS batch jobs and SSH/MPI launches so lateral movement across compute nodes is linked into unified cross-node stories under ε-differential privacy.
3. **Multi-Scale Sub-Threshold CUSUM for Ultra-Slow Ransomware:** Addressing residual gaps `G2`–`G3` by tracking cumulative file-entropy shifts and directory-traversal surprisal over multi-day horizons.
4. **Hardware-Backed Policy Signing & Verifiable Logs:** Binding Ed25519 policy keys and device HMAC keys to TPM 2.0 / HSM hardware roots of trust so even a privileged host compromise cannot forge policy weakenings or retroactively rewrite hash-chained audit ledgers.
5. **Offline Adversarial Search in the Replay Lab:** A search procedure (for example a reinforcement-learning attacker) that mutates event streams to find variants the deterministic monitor allows, replacing hand-written mutation dimensions; the monitor itself stays deterministic, and each finding enters the pre-registered cycle as a design finding.
6. **Longitudinal University Pilot (Phase 9):** Measuring real-world weekly hold rates per user, service-desk resolution times, per-syscall latency overhead, and contract-learning convergence across heterogeneous research workstations and web services.

7 Conclusion

We presented **Refmon** (*Provenance Gate v2*, specification `v2.0`), a deterministic, privacy-preserving reference monitor that unifies dynamic provenance tracking, cryptographically signed role contracts, six-engine cross-view rootkit inspection, and cost-sensitive Bayesian log-odds calculus. By stripping raw payloads at an in-memory `Observe` boundary and judging causal stories through three interpretable questions—Origin, Contract, and Effect—Refmon stops ClickFix social-engineering pastes, zero-day server RCEs, web shells, ransomware bursts, AI prompt-injection tool abuse, and stealth preload/LKM rootkits without machine-learning opacity or cleartext telemetry collection. Across 46 golden scenarios, nine pre-registered variant seeds, and live unprivileged cluster barrier testing, Refmon `v2.0` achieves 100% golden accuracy, 98.4% mutated attack containment on a fresh seed, a 0.0% neutral false-block rate (down from 2.0% for `v1.3`), and provable mathematical guarantees on monotonicity, Bayes risk optimality, bounded trust, and zero raw data leakage.

8 AI Disclosure

In accordance with academic transparency and research-integrity standards, the authors disclose the structured use of generative AI coding and research assistants during the development, verification, and documentation of Refmon:

1. **Phased Implementation Under Human Specification:** Core pipeline modules (`Phases 0-7`, tags `v1.0`–`v1.3`) were developed iteratively using *Anthropic Claude Code* governed by a human-authored specification (`CLAUDE.md`). Every phase required deterministic unit tests, Hypothesis property-based tests, and human review prior to progression. The pre-registered rounds `v1.5`–`v2.0` (design decisions A60–A67), the hardening of the live API (per-daemon token, `Host` check, owner-only work directory, random device key), the command-intent analyser, the diagram book, and the `v2.0` revision of this manuscript were likewise produced with Claude Code under the same specification and tests. Crucially, **no AI or machine-learning model exists inside the Refmon runtime**: all runtime security decisions are purely deterministic mathematical functions of counted baselines and signed policy parameters.
2. **Extended Rootkit Sensor, Cluster Barrier, and Manuscript Preparation:** *Google DeepMind Antigravity* assisted in engineering the six-engine read-only cross-view rootkit scanner (`src/refmon/rootkit.py`), the unprivileged live cluster barrier and light-theme telemetry interface (`src/refmon/live.py`, `src/refmon/live_ui.py`), their integration test suites (`tests/test_rootkit.py`, `tests/test_live.py`), architectural diagrams, and the LaTeX typesetting of this manuscript and its mathematical proofs.
3. **Safeguards Against Overfitting and Hallucination:** All signal weights were frozen prior to benchmark runs; mutation evaluations (`v1.0`–`v2.0`) used pre-registered random seeds recorded before structural changes; cryptographic policy keys remain exclusively human-held; and every theorem, equation, and empirical figure in this paper was verified against executable code and reproducible test outputs (`make reproduce`, `pytest`, and `refmon audit`).

References

[1] J. P. Anderson, "Computer security technology planning study," Electronic Systems Division, Air Force Systems Command, Tech. Rep. ESD-TR-73-51, Vol. II, Oct. 1972.

[2] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, May 1976.

[3] A. Wald, "Sequential tests of statistical hypotheses," *The Annals of Mathematical Statistics*, vol. 16, no. 2, pp. 117–186, Jun. 1945.

[4] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, Jun. 1954.

[5] G. Lorden, "Procedures for reacting to a change in distribution," *The Annals of Mathematical Statistics*, vol. 42, no. 6, pp. 1897–1908, Dec. 1971.

[6] I. J. Good, "The population frequencies of species and the estimation of population parameters," *Biometrika*, vol. 40, no. 3/4, pp. 237–264, Dec. 1953.

[7] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA: Morgan Kaufmann, 1988.

[8] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970.

[9] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," in *Advances in Cryptology — CRYPTO '96*, LNCS vol. 1109, pp. 1–15, Springer, 1996.

[10] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, Sep. 2012.

[11] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. Theory of Cryptography Conf. (TCC)*, LNCS vol. 3876, pp. 265–284, Springer, 2006.

[12] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557–570, Oct. 2002.

[13] J. Newsome and D. X. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in *Proc. NDSS*, Feb. 2005.

[14] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eyers, M. Seltzer, and J. Bacon, "Practical whole-system provenance capture," in *Proc. ACM SoCC*, pp. 405–418, Sep. 2017.

[15] M. N. Hossain, S. M. Milajerdi, J. Wang, B. Eshete, R. Gjomemo, R. Sekar, S. Stolper, and V. N. Venkatakrishnan, "SLEUTH: Real-time attack scenario reconstruction from COTS audit data," in *Proc. USENIX Security Symp.*, pp. 487–504, Aug. 2017.

[16] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *Proc. IEEE Symp. Security and Privacy (S&P)*, pp. 1137–1152, May 2019.

[17] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime provenance-based detector for advanced persistent threats," in *Proc. NDSS*, Feb. 2020.

[18] S. Smalley, C. Vance, and W. Salamon, "Implementing SELinux as a Linux Security Module," NAI Labs Report #01-043, Dec. 2001.

[19] M. Bauer, "Paranoid penguin: An introduction to Novell AppArmor," *Linux Journal*, vol. 2006, no. 148, p. 13, Aug. 2006.

[20] J. Corbet, "Seccomp filter (Mode 2 BPF syscall filtering)," Linux Kernel Documentation (`Documentation/userspace-api/seccomp_filter.rst`), 2012.

[21] Y.-M. Wang, D. Beck, B. Vo, R. Roussev, and C. Verbowski, "Detecting stealth software with Strider GhostBuster," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, pp. 368–377, Jun. 2005.

[22] T. Garfinkel and M. Rosenblum, "A virtual machine introspection based architecture for intrusion detection," in *Proc. NDSS*, pp. 191–206, Feb. 2003.

[23] J. Jung, V. Paxson, A. W. Berger, and H. Balakrishnan, "Fast portscan detection using sequential hypothesis testing," in *Proc. IEEE Symp. Security and Privacy (S&P)*, pp. 211–225, May 2004.

[24] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec Workshop*, pp. 79–90, Nov. 2023.

[25] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix (T1204.004 User Execution: Malicious Copy and Paste)," Technical Knowledge Base, 2024–2026.

[26] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, Jun. 1927.

[27] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, Jun. 1947.

A Proofs of Provenance Decay and Noisy-OR Fusion

Proposition A.1 (Monotonicity and Invariance of Label Decay)**.** *Let $s_\ell(t) = s_0 \cdot 2^{-(t-t_0)/T_{1/2}}$ for $s_0 \in [0,1]$, $T_{1/2} > 0$, and $t \geq t_0$. Then:*

1. *$s_\ell(t) \in [0, s_0] \subseteq [0,1]$ for all $t \geq t_0$;*
2. *$s_\ell(t)$ is strictly decreasing in t whenever $s_0 > 0$, with $s_\ell(t_0 + kT_{1/2}) = 2^{-k} s_0$;*
3. *Under edge attenuation $a_e \in (0,1]$, any d-hop causal path $p = (e_1, \ldots, e_d)$ satisfies $s_{leaf}(t) = s_0\big(\prod_{r=1}^{d} a_{e_r}\big) 2^{-\Delta t/T_{1/2}} \leq s_\ell(t)$.*

Proof. Since $T_{1/2} > 0$ and $\Delta t = t - t_0 \geq 0$, we have $-\Delta t/T_{1/2} \leq 0$, so $0 < 2^{-\Delta t/T_{1/2}} \leq 1$, proving $0 \leq s_\ell(t) \leq s_0 \leq 1$. Differentiating with respect to t yields

$$\frac{\mathrm{d}s_\ell(t)}{\mathrm{d}t} = -\frac{\ln 2}{T_{1/2}}\, s_0\, 2^{-(t-t_0)/T_{1/2}} < 0 \quad (\forall s_0 > 0),$$

establishing strict temporal monotonicity, and direct substitution of $\Delta t = kT_{1/2}$ gives $s_\ell(t_0 + kT_{1/2}) = 2^{-k} s_0$. Finally, because $a_{e_r} \in (0,1]$ for every edge e_r, induction over path length d yields $0 < \prod_{r=1}^{d} a_{e_r} \leq 1$, whence $s_{\text{leaf}}(t) \leq s_\ell(t)$. □

Theorem A.2 (Algebraic and Dominance Properties of Noisy-OR)**.** *For any finite multiset of label strengths $\mathbf{s} = (s_1, \ldots, s_m) \in [0,1]^m$, define $\mathrm{NOR}(\mathbf{s}) = 1 - \prod_{i=1}^{m}(1 - s_i)$ (with empty product 1). Then:*

1. ***Range closure:*** *$\mathrm{NOR}(\mathbf{s}) \in [0,1]$;*
2. ***Dominance:*** *$\max_{1 \leq i \leq m} s_i \leq \mathrm{NOR}(\mathbf{s}) \leq \min\big(1, \sum_{i=1}^{m} s_i\big)$;*
3. ***Commutative monoid:*** *NOR is symmetric, associative under binary composition $x \oplus y = 1 - (1-x)(1-y) = x + y - xy$, has identity element 0, and absorbing element 1;*
4. ***Coordinate-wise monotonicity:*** *If $s_i \leq s_i'$ for all i, then $\mathrm{NOR}(\mathbf{s}) \leq \mathrm{NOR}(\mathbf{s}')$.*

Proof. (1) Since $s_i \in [0,1]$, each factor satisfies $1 - s_i \in [0,1]$, so $P = \prod_{i=1}^{m}(1 - s_i) \in [0,1]$ and $1 - P \in [0,1]$.

(2) Fix an arbitrary index $k \in \{1, \ldots, m\}$. Because $\prod_{i \neq k}(1 - s_i) \leq 1$ and $1 - s_k \geq 0$, we obtain

$$\prod_{i=1}^{m}(1 - s_i) = (1 - s_k) \prod_{i \neq k}(1 - s_i) \leq 1 - s_k$$
$$\implies \mathrm{NOR}(\mathbf{s}) \geq s_k.$$

Taking the maximum over k yields $\mathrm{NOR}(\mathbf{s}) \geq \max_i s_i$. For the upper bound, induction on m using Weierstrass's inequality $\prod_{i=1}^{m}(1 - s_i) \geq 1 - \sum_{i=1}^{m} s_i$ gives $\mathrm{NOR}(\mathbf{s}) \leq \sum_{i=1}^{m} s_i$, and combining with (1) gives $\min(1, \sum_i s_i)$.

(3) Symmetry follows from commutativity of real multiplication. For associativity, $(x \oplus y) \oplus z = 1 - (1 - [1 - (1-x)(1-y)])(1-z) = 1 - (1-x)(1-y)(1-z) = x \oplus (y \oplus z)$. Moreover, $x \oplus 0 = 1 - (1-x) \cdot 1 = x$ and $x \oplus 1 = 1 - (1-x) \cdot 0 = 1$.

(4) If $s_i \leq s_i'$, then $0 \leq 1 - s_i' \leq 1 - s_i$ for each i, so $\prod_{i=1}^{m}(1 - s_i') \leq \prod_{i=1}^{m}(1 - s_i)$, whence $\mathrm{NOR}(\mathbf{s}) \leq \mathrm{NOR}(\mathbf{s}')$. □

B Derivation of Bayes-Optimal Thresholds and Contract Gate

Theorem B.1 (Bayes Minimal-Risk Log-Odds Thresholds)**.** *Let $Y \in \{0,1\}$ denote whether a story S is benign ($Y = 0$) or an attack ($Y = 1$), and let $p = \mathbb{P}(Y = 1 \mid S) = (1 + e^{-L})^{-1}$ be the posterior attack probability corresponding to log-odds $L = \ln \frac{p}{1-p}$. Suppose:*

- *Action a_{allow} incurs loss $C_{miss} > 0$ if $Y = 1$ and 0 if $Y = 0$;*
- *Action a_{block} incurs loss 0 if $Y = 1$ and $C_{fb} > 0$ if $Y = 0$;*
- *Action a_{hold} incurs human-review cost $C_{hold} > 0$ if $Y = 0$, and catches a true attack ($Y = 1$) with probability $h \in (0,1]$ (leaving residual miss cost $(1-h)C_{miss}$; when $C_{hold} \ll C_{miss}$ we neglect the $O(C_{hold})$ operational overhead under $Y = 1$).*

Then:

1. *Pairwise preference $R(a_{block} \mid S) \leq R(a_{allow} \mid S)$ holds iff*
$$L \geq \tau_{block} \triangleq \ln \frac{C_{fb}}{C_{miss}};$$
2. *Pairwise preference $R(a_{hold} \mid S) \leq R(a_{allow} \mid S)$ holds iff*
$$L \geq \tau_{hold} \triangleq \ln \frac{C_{hold}}{h \cdot C_{miss}};$$
3. *A non-empty `HOLD` interval $[\tau_{hold}, \tau_{block})$ exists iff $C_{hold} < h \cdot C_{fb}$.*

Proof. The conditional Bayes risks of the three actions given posterior $p \in (0,1)$ are:

$$R(a_{\text{allow}} \mid S) = p\, C_{\text{miss}},$$
$$R(a_{\text{block}} \mid S) = (1-p)\, C_{\text{fb}},$$
$$R(a_{\text{hold}} \mid S) = p(1-h)C_{\text{miss}} + (1-p)C_{\text{hold}}.$$

For (1), $R(a_{\text{block}} \mid S) \leq R(a_{\text{allow}} \mid S)$ iff $(1-p)C_{\text{fb}} \leq p\, C_{\text{miss}}$, which rearranges to

$$\frac{p}{1-p} \geq \frac{C_{\text{fb}}}{C_{\text{miss}}} \iff L = \ln \frac{p}{1-p} \geq \ln \frac{C_{\text{fb}}}{C_{\text{miss}}} = \tau_{\text{block}}.$$

For (2), $R(a_{\text{hold}} \mid S) \leq R(a_{\text{allow}} \mid S)$ iff

$$p(1-h)C_{\text{miss}} + (1-p)C_{\text{hold}} \leq p\, C_{\text{miss}}$$
$$\iff (1-p)C_{\text{hold}} \leq p\, h\, C_{\text{miss}}.$$

Dividing by $(1-p)hC_{\text{miss}} > 0$ and taking natural logarithms yields

$$\frac{p}{1-p} \geq \frac{C_{\text{hold}}}{h \cdot C_{\text{miss}}} \iff L \geq \ln \frac{C_{\text{hold}}}{h \cdot C_{\text{miss}}} = \tau_{\text{hold}}.$$

For (3), by strict monotonicity of $\ln(\cdot)$, $\tau_{\text{hold}} < \tau_{\text{block}} \iff \frac{C_{\text{hold}}}{hC_{\text{miss}}} < \frac{C_{\text{fb}}}{C_{\text{miss}}} \iff C_{\text{hold}} < h\,C_{\text{fb}}$. Substituting Refmon's default policy costs ($C_{\text{hold}} = 5$, $h = 0.7$, $C_{\text{fb}} = 20$) gives $5 < 14$, yielding $\tau_{\text{hold}} = \ln(5/700) = -4.9416 < \tau_{\text{block}} = \ln(20/1000) = -3.9120$. □

Corollary B.2 (Sensitivity Directionality)**.** *$\partial\tau_{block}/\partial C_{fb} = 1/C_{fb} > 0$, $\partial\tau_{block}/\partial C_{miss} = -1/C_{miss} < 0$, and $\partial\tau_{hold}/\partial h = -1/h < 0$. Thus raising false-block penalties raises the evidence bar for blocking, whereas raising missed-attack cost lowers both thresholds monotonically.*

C Proof of Posterior Monotonicity in `v1.3` (Resolution of F8)

Theorem C.1 (Monotonicity of `v1.3` Calculus vs. Non–Monotonicity of `v1.2`)**.** *Fix a story S with non-empty Contract or Effect signals having positive max weights $W_C = \max_{j\in\mathcal{S}_C} w_j > 0$ and $W_E = \max_{k\in\mathcal{S}_E} w_k \geq 0$, origin weight $w_{ori} \geq 0$, and origin threshold $\theta \in (0,1)$.*

1. *Under the legacy `v1.2` rule $L_{v1.2}(s) = L_0 + w_{ori}s\mathbf{1}[s \geq \theta] + \big(s\mathbf{1}[s \geq \theta] + \mathbf{1}[s < \theta]\big)(W_C + W_E)$, $L_{v1.2}(s)$ is **not** non-decreasing on $[0,1]$ whenever $W_C + W_E > \frac{\theta}{1-\theta} w_{ori}$.*
2. *Under the `v1.3` rule $L_{v1.3}(s) = L_0 + w_{ori}s\mathbf{1}[s \geq \theta] + W_C + W_E$, $L_{v1.3}(s)$ is monotonically non-decreasing over all $s \in [0,1]$ and under arbitrary signal additions with $w_j \geq 0$.*

Proof. (1) Evaluate the left and right limits of $L_{\text{v1.2}}(s)$ at $s = \theta$:

$$\lim_{s\uparrow\theta} L_{\text{v1.2}}(s) = L_0 + W_C + W_E,$$
$$L_{\text{v1.2}}(\theta) = L_0 + \theta w_{\text{ori}} + \theta(W_C + W_E).$$

Hence $L_{\text{v1.2}}(\theta) - \lim_{s\uparrow\theta} L_{\text{v1.2}}(s) = \theta w_{\text{ori}} - (1-\theta)(W_C + W_E)$. For Refmon's defaults ($\theta = 0.5$, $w_{\text{ori}} = 0.8$), whenever $W_C + W_E > 0.8$ (which holds for *every* Contract + Effect pair in Table 1, e.g., $W_C + W_E = 1.0 + 2.5 = 3.5$), we have $L_{\text{v1.2}}(0.5) - L_{\text{v1.2}}(0.499) = 0.4 - 1.75 = -1.35 < 0$, proving a downward jump where stronger provenance produced weaker evidence.

(2) In `v1.3`, write $g(s) = w_{\text{ori}}s\mathbf{1}[s \geq \theta]$. For $0 \leq s_1 \leq s_2 < \theta$, $g(s_1) = g(s_2) = 0$. For $0 \leq s_1 < \theta \leq s_2 \leq 1$, since $w_{\text{ori}} \geq 0$ and $s_2 \geq \theta > 0$, $g(s_1) = 0 \leq w_{\text{ori}}s_2 = g(s_2)$. For $\theta \leq s_1 \leq s_2 \leq 1$, $g(s_2) - g(s_1) = w_{\text{ori}}(s_2 - s_1) \geq 0$. Since W_C and W_E are independent of s (apart from D23's indicator $\mathbf{1}[s \geq \theta]$ which only *adds* positive allowed-effect weights when s crosses θ), $L_{\text{v1.3}}(s_1) \leq L_{\text{v1.3}}(s_2)$ for all $0 \leq s_1 \leq s_2 \leq 1$. Furthermore, adding any signal j^* with $w_{j^*} \geq 0$ to question set $\mathcal{S}_q$ satisfies $\max_{j\in\mathcal{S}_q\cup\{j^*\}} w_j s_j \geq \max_{j\in\mathcal{S}_q} w_j s_j$, so $L_{\text{v1.3}}$ never decreases when evidence is added. □

D Proofs of Laplace Surprisal and Good-Turing Unseen Mass

Proposition D.1 (Properness and Surprisal Bounds of Laplace Role Model)**.** *Let $n_1, \ldots, n_K \geq 1$ be observed counts summing to $N = \sum_{e=1}^{K} n_e$, and define $\hat{p}(e) = \frac{n_e+\alpha}{N+\alpha(K+1)}$ for $e \in \{1,\ldots,K\}$ and $\hat{p}(\text{unseen}) = \frac{\alpha}{N+\alpha(K+1)}$ with $\alpha > 0$. Then $\hat{p}$ is a valid probability distribution on $\{1,\ldots,K,\text{unseen}\}$, and unseen-event surprisal satisfies $I(\text{unseen}) = \log_2\big(\frac{N}{\alpha} + K + 1\big)$.*

Proof. Every probability is strictly positive since $n_e \geq 0$ and $\alpha > 0$. Summing over all $K+1$ outcomes gives

$$\sum_{e=1}^{K} \frac{n_e + \alpha}{N + \alpha(K+1)} + \frac{\alpha}{N + \alpha(K+1)}$$
$$= \frac{\sum_{e=1}^{K} n_e + K\alpha + \alpha}{N + \alpha(K+1)} = \frac{N + \alpha(K+1)}{N + \alpha(K+1)} = 1.$$

Taking $-\log_2 \hat{p}(\text{unseen})$ yields $\log_2((N+\alpha(K+1))/\alpha) = \log_2(N/\alpha + K + 1)$. In particular, for $\alpha = 1$, $I(\text{unseen}) \geq 12$ bits requires $N + K + 1 \geq 2^{12} = 4096$, automatically preventing surprisal false alarms on small samples even before the Good-Turing check. □

Theorem D.2 (Unbiasedness of Good-Turing Unseen Probability Estimator [6])**.** *Suppose N i.i.d. events are drawn from an unknown discrete distribution $\{p_e\}_{e\in\mathcal{U}}$ over a countable alphabet $\mathcal{U}$. Let N_e be the count of kind e in the first N draws, $U_N = \sum_{e\in\mathcal{U}} p_e\mathbf{1}[N_e = 0]$ be the true random probability mass of kinds unseen in N draws, and $N_1^{(N+1)} = \sum_{e\in\mathcal{U}} \mathbf{1}[N'_e = 1]$ be the number of singletons in a sample of size $N+1$. Then:*

$$\mathbb{E}[U_N] = \frac{\mathbb{E}\big[N_1^{(N+1)}\big]}{N+1} \approx \frac{\mathbb{E}[N_1^{(N)}]}{N}.$$

Proof. By linearity of expectation and $N_e \sim \text{Binomial}(N, p_e)$,

$$\mathbb{E}[U_N] = \sum_{e\in\mathcal{U}} p_e\mathbb{P}(N_e = 0) = \sum_{e\in\mathcal{U}} p_e(1-p_e)^N.$$

Conversely, in $N+1$ i.i.d. draws, $\mathbb{P}(N'_e = 1) = (N+1)p_e(1-p_e)^N$. Summing over $e \in \mathcal{U}$ and dividing by $N+1$ gives

$$\frac{\mathbb{E}\big[N_1^{(N+1)}\big]}{N+1} = \frac{1}{N+1}\sum_{e\in\mathcal{U}} (N+1)p_e(1-p_e)^N$$
$$= \sum_{e\in\mathcal{U}} p_e(1-p_e)^N = \mathbb{E}[U_N].$$

For $N \gg 1$, replacing $N+1$ by N yields $\hat{p}_{\text{unseen}} = N_1/N$ with $O(1/N)$ bias. □

E Proofs of Statistical Rate and CUSUM Drift Properties

Proposition E.1 (Numerical Exactness of Upper-Tail Poisson Recurrence)**.** *For $X \sim \text{Poisson}(\lambda)$ with $\lambda > 0$ and integer $k \geq 1$, the upper tail $T(k,\lambda) = \sum_{i=k}^{\infty} t_i$ with $t_k = \exp(-\lambda + k\ln\lambda - \ln\Gamma(k+1))$ and recurrence $t_{i+1} = t_i \cdot \frac{\lambda}{i+1}$ converges strictly monotonically for $i \geq \lceil\lambda\rceil$ with relative truncation error below machine epsilon ε_{mach} once $t_i \leq \varepsilon_{mach} \sum_{j=k}^{i} t_j$.*

Proof. Computing $\ln t_k$ via $\text{lgamma}(k+1) = \ln(k!)$ avoids catastrophic cancellation in $1 - \sum_{i=0}^{k-1} t_i$ when $T(k,\lambda) \approx 10^{-12} \ll \varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$ (where complement subtraction returns 0.0). For all $i \geq \max(k, \lceil 2\lambda \rceil)$, $t_{i+1}/t_i = \lambda/(i+1) \leq 1/2$, so the omitted tail satisfies $\sum_{j=i+1}^{\infty} t_j \leq t_{i+1} \sum_{r=0}^{\infty} 2^{-r} = 2t_{i+1} \leq t_i$, bounding relative error by $\varepsilon_{\text{mach}}$. □

Proposition E.2 (Negative Drift Under Benign Null and Poisoning Immunity)**.** *Let $z_t = (x_t - \mu)/\sigma$ have $\mathbb{E}_0[z_t] = 0$ under benign traffic and $\mathbb{E}_1[z_t] = \delta > k_c > 0$ under slow exfiltration. Then the one-sided CUSUM $S_t = \max(0, S_{t-1} + z_t - k_c)$ has strictly negative pre-reflection increment drift $\mathbb{E}_0[z_t - k_c] = -k_c < 0$ under $\mathcal{H}_0$ (preventing unbounded random-walk drift) and strictly positive drift $\mathbb{E}_1[z_t - k_c] = \delta - k_c > 0$ under $\mathcal{H}_1$, matching the Gaussian log-likelihood ratio test for shift $\delta = 2k_c$ [4, 5]. Moreover, when EMA updates $\hat{\lambda}_t = (1-\beta)\hat{\lambda}_{t-1} + \beta y_t \cdot \mathbf{1}[\neg \textit{incident}]$ are gated on J.decision = `ALLOW`, any sequence of anomalous bursts during an open incident leaves $\hat{\lambda}_t \equiv \hat{\lambda}_{t_{open}}$, preventing baseline inflation.*

Proof. Under Gaussian $z_t \sim \mathcal{N}(\delta, 1)$ vs. $\mathcal{N}(0,1)$, the single-step log-likelihood ratio is $\ln \frac{\phi(z_t - \delta)}{\phi(z_t)} = \delta z_t - \frac{\delta^2}{2} = \delta\big(z_t - \frac{\delta}{2}\big)$. Setting $k_c = \delta/2 > 0$ scales Page's SPRT recursion directly into Eq. (17). Under $\mathcal{H}_0$, $\mathbb{E}_0[z_t - k_c] = -k_c < 0$; under $\mathcal{H}_1$ with $\delta > k_c$, $\mathbb{E}_1[z_t - k_c] = \delta - k_c > 0$. Finally, during $[t_{\text{open}}, t_{\text{close}}]$, $\mathbf{1}[\neg\text{incident}] = 0 \implies \hat{\lambda}_t = \hat{\lambda}_{t_{\text{open}}}$ by trivial induction. □

F Proofs of Differential Privacy and Zero-FP Bloom Filtering

Theorem F.1 (ε-Differential Privacy of Aggregate Fleet Counts)**.** *Let D, D' be neighboring device-telemetry datasets differing in the state of at most one device, and let $f(D) \in \mathbb{N}$ count the number of devices experiencing a given rule or tactic ($\Delta f = \max_{D \sim D'} |f(D) - f(D')| = 1$). Then the mechanism $\mathcal{M}(D) = f(D) + \eta$ where η is sampled via inverse transform $F^{-1}(u) = -\frac{1}{\varepsilon}\,\text{sgn}(u - \frac{1}{2}) \ln\big(1 - 2|u - \frac{1}{2}|\big)$ for $u \sim \text{Uniform}(0,1)$ satisfies pure ε-differential privacy.*

Proof. First, for $y \geq 0$, $\mathbb{P}(F^{-1}(u) \geq y) = \mathbb{P}\big(u - \frac{1}{2} \geq \frac{1}{2}(1 - e^{-\varepsilon y})\big) = \frac{1}{2}e^{-\varepsilon y}$; by symmetry around 0, η has density $p_\eta(z) = \frac{\varepsilon}{2}\exp(-\varepsilon|z|)$, i.e., $\eta \sim \text{Lap}(0, 1/\varepsilon)$. Second, for any measurable output $z \in \mathbb{R}$ and neighboring D, D' with $|f(D) - f(D')| \leq \Delta f = 1$, the reverse triangle inequality $|z - f(D')| - |z - f(D)| \leq |f(D) - f(D')| \leq 1$ gives:

$$\frac{p_{\mathcal{M}(D)}(z)}{p_{\mathcal{M}(D')}(z)} = \frac{\frac{\varepsilon}{2}\exp(-\varepsilon|z - f(D)|)}{\frac{\varepsilon}{2}\exp(-\varepsilon|z - f(D')|)} = \exp\Big(\varepsilon\big(|z - f(D')| - |z - f(D)|\big)\Big) \leq e^{\varepsilon}. \;\square$$

Theorem F.2 (Optimal Sizing and Zero Operational False Positives of Confirmed Bloom Threat Lists)**.** *For n threat entries inserted into an m-bit array using k_h independent hash positions, the asymptotic filter false-positive probability $p(k_h) = \big(1 - e^{-k_h n/m}\big)^{k_h}$ is minimized at $k_h^* = \frac{m}{n}\ln 2$, requiring $m = \lceil -n \ln p_{fp}/(\ln 2)^2 \rceil$ bits for target hint rate p_{fp}. Moreover, two-stage confirmation $H(x) = \mathbf{1}[\mathcal{B}(x) = 1 \wedge d(x) \in \mathcal{D}_{exact}]$ achieves zero false negatives relative to $\mathcal{D}_{exact}$ and false-positive probability $\leq 2^{-256}$ (HMAC-SHA256 collision bound).*

Proof. Let $g(k_h) = \ln p(k_h) = k_h \ln(1 - e^{-k_h n/m})$. Setting $x = e^{-k_h n/m} \in (0,1)$ gives $k_h = -\frac{m}{n}\ln x$ and $g = -\frac{m}{n}\ln x \ln(1-x)$. Differentiating with respect to x:

$$\frac{\mathrm{d}g}{\mathrm{d}x} = -\frac{m}{n}\left(\frac{\ln(1-x)}{x} - \frac{\ln x}{1-x}\right) = 0 \iff x = 1 - x \iff x = \frac{1}{2}.$$

Substituting $x = 1/2$ yields $k_h^* = \frac{m}{n}\ln 2$ and $p(k_h^*) = (1/2)^{(m/n)\ln 2} = e^{-m(\ln 2)^2/n} \leq p_{\text{fp}} \iff m \geq -n\ln p_{\text{fp}}/(\ln 2)^2$. Finally, because a Bloom filter has zero false negatives ($d(x) \in \mathcal{D}_{\text{exact}} \implies \mathcal{B}(x) = 1$), $H(x) = 1 \iff d(x) \in \mathcal{D}_{\text{exact}}$; for $x \notin \mathcal{D}_{\text{bad}}$, $d(x) \in \mathcal{D}_{\text{exact}}$ occurs only on an HMAC-SHA256 collision across n entries ($\leq n \cdot 2^{-256}$ under PRF security [9]). □

G Bounds of the Trust Mark (`v2.0`, A65)

Proposition G.1 (Trust Never Outlives or Outreaches a Label)**.** *Let process p ingest trusted-root content at t_p, let $s_{floor} \in (0,1)$ be the label alive floor, and let $\text{trust}_p(t)$ be defined by Eq. (8). Then:*

1. ***Lifetime:*** *$\text{trust}_p(t) = 1$ implies $t - t_p \leq T_{1/2}\log_2(1/s_{floor})$, the lifetime of a non-persistent label without contract evidence created at t_p;*
2. ***Cancellation:*** *if any label in $\text{ever}(p)$ is alive at t, then $\text{trust}_p(t) = 0$, whatever its strength relative to θ_{ori};*
3. ***Reach:*** *the mark reaches only p and the processes spawned in its lineage; a channel role that ingests*

trusted content does not acquire it, and no file or persistence item carries it, so a startup entry written under trust starts a later story without trust;

4. ***Precedence:*** *no story containing a self-tamper, canary or protected-asset flag is allowed by the mark.*

Proof. (1) The first factor of Eq. (8) is 1 iff $2^{-(t-t_p)/T_{1/2}} \geq s_{\text{floor}} \iff t - t_p \leq T_{1/2} \log_2(1/s_{\text{floor}})$, which by Eq. (6) with $s_0 = 1$ is exactly the time at which such a label falls below the floor (Proposition A.1). With $T_{1/2} = 30$ min and $s_{\text{floor}} = 0.02$ this is $30 \log_2 50 \approx 169$ min. (2) The second factor is 0 by definition. (3) The mark is set on an ingesting process only when its role is not a channel role, and copied only when a new process is created with p (or a descendant) as parent; rule 5 (files and persistence items) copies labels, never the mark. (4) `Judge` evaluates the overrides in the fixed order self-tamper $\succ$ canary $\succ$ protected asset $\succ$ trusted root and returns the first that applies, so the trusted-root override is reached only when none of the blocking overrides is present. □

H Proof of Multi-Pass Rootkit Churn Suppression

Proposition H.1 (Exponential Suppression of Transient OS Race Discrepancies)**.** *Suppose during pass $r \in \{1, \ldots, m\}$ separated by $\Delta\tau$ exceeding typical syscall directory-walk latency, a benign short-lived entity x (such as an exiting worker PID or unloading kernel module) produces a transient cross-view race $x \in \mathcal{V}_{low}^{(r)} \setminus \mathcal{V}_{high}^{(r)}$ with probability $p_{race} \ll 1$ on Pass 1 and conditional persistence probability $q_{persist} \ll 1$ on each subsequent pass, whereas a stealth rootkit actively hiding x^* satisfies $x^* \in \mathcal{V}_{low}^{(r)} \setminus \mathcal{V}_{high}^{(r)}$ with probability $p_{tp} \approx 1$ on every pass. Then for $\Delta_{pers} = \bigcap_{r=1}^{m} (\mathcal{V}_{low}^{(r)} \setminus \mathcal{V}_{high}^{(r)})$:*

$$\mathbb{P}(x \in \Delta_{pers} \mid \textit{benign}) = p_{race} \cdot q_{persist}^{m-1},$$

$$\ln \frac{\mathbb{P}(x \in \Delta_{pers} \mid \textit{rootkit})}{\mathbb{P}(x \in \Delta_{pers} \mid \textit{benign})} = \ln \frac{p_{tp}}{p_{race}} + (m-1) \ln \frac{1}{q_{persist}}.$$

Proof. By the chain rule of conditional probability under $\mathcal{H}_0$ (benign OS churn) across passes $1, \ldots, m$,

$$\mathbb{P}\left(\bigcap_{r=1}^{m} E_r \,\middle|\, \mathcal{H}_0 \right) = \mathbb{P}(E_1 \mid \mathcal{H}_0) \prod_{r=2}^{m} \mathbb{P}(E_r \mid E_{1..r-1}, \mathcal{H}_0)$$
$$= p_{\text{race}}\, q_{\text{persist}}^{m-1}.$$

Under a persistent rootkit hook (e.g., an LKM unlinked from `modules` list but still mounted in `/sys/module`, or `getdents64` filtering PID x^*), E_r holds deterministically across passes ($\mathbb{P}(\bigcap_{r=1}^{m} E_r \mid \text{rootkit}) = p_{\text{tp}}$). Taking the natural log-likelihood ratio yields $\ln(p_{\text{tp}}/p_{\text{race}}) + (m-1)\ln(1/q_{\text{persist}})$, and applying the policy cap $w_{\max}$ yields Eq. (19). For $p_{\text{race}} = 0.02$, $q_{\text{persist}} = 0.05$, and $m = 2$, benign race probability drops from 2×10^{-2} to 10^{-3} (20× reduction) in a single 50 ms interval. □